Grupo 12

GAMMA LAB 2.0

Juan Felipe Romero Ortiz
Laura Juliana Garzón Arias
Sebastián Almanza Galvis
Xiomara Herrera Suárez

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
BOGOTÁ, D.C.
2026

GAMMA LAB 2.0

Autores:

Juan Felipe Romero Ortiz
Laura Juliana Garzón Arias
Sebastián Almanza Galvis
Xiomara Herrera Suárez

MEMORIA DE PROYECTO DE GRADO REALIZADO PARA CUMPLIR UNO DE LOS REQUISITOS PARA EL TÍTULO EN INGENIERÍA DE SISTEMAS

Director

Ing. Cecile Eugine Gauthier Umaña

Jurados del Trabajo Final de Grado

Ing. Nombre del Primer Jurado
Ing. Nombre del Segundo Jurado

PONTIFICIA UNIVERSIDAD JAVERIANA

FACULTAD DE INGENIERÍA

PROGRAMA DE INGENIERÍA DE SISTEMAS

BOGOTÁ, D.C.

Noviembre, 2026

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS

Rector de la Pontificia Universidad Javeriana

Nombre del Rector, S.J.

Decano de la Facultad de Ingeniería

Ing. Nombre del Decano de la Facultad

Director del Departamento de Ingeniería de Sistemas

Ing. Nombre del Director de Departamento, Ph.D.

Directora de Carrera de Ingeniería de Sistemas

Ing. Nombre de la Directora de Carrera

Artículo 23 de la Resolución No. 1 de Junio de 1946

"La Universidad no se hace responsable de los conceptos emitidos por sus alumnos en sus proyectos de grado. Sólo velará porque no se publique nada contrario al dogma y la moral católica y porque no contengan ataques o polémicas puramente personales. Antes bien, que se vean en ellos el anhelo de buscar la verdad y la Justicia"

AGRADECIMIENTOS

Juan Felipe Romero Ortiz:

[Escriba aquí los agradecimientos del primer autor. Puede usar varios párrafos; cada uno inicia con sangría automática.]

Laura Juliana Garzón Arias:

[Escriba aquí los agradecimientos del segundo autor.]

Sebastián Almanza Galvis:

[Escriba aquí los agradecimientos del tercer autor.]

Xiomara Herrera Suárez:

[Escriba aquí los agradecimientos del cuarto autor.]

Índice

1. INTRODUCCIÓN **1**

2. DESCRIPCIÓN GENERAL **2**
2.1. Oportunidad y problema . . . 2
2.1.1. Contexto del Problema . . . 2
2.1.2. Formulación del Problema . . . 3
2.1.3. Propuesta de Solución . . . 4
2.1.4. Justificación de la solución . . . 6
2.2. Descripción del Proyecto . . . 6
2.2.1. Objetivo General . . . 6
2.2.2. Objetivos Específicos . . . 7
2.3. Entregables, estándares utilizados y justificación . . . 7

3. CONTEXTO DEL PROYECTO **8**
3.1. Trasfondo . . . 8
3.2. Alternativas de solución . . . 10
3.2.1. Alternativa 1 . . . 10
3.2.2. Alternativa 2 . . . 10
3.2.3. Alternativa 3 . . . 10
3.2.4. Alternativa 4 . . . 11
3.3. Comparación de alternativas . . . 11

4. ANÁLISIS DEL PROBLEMA **12**
4.1. Requerimientos . . . 12
4.1.1. Requerimientos Funcionales . . . 12
4.1.2. Requerimientos No Funcionales . . . 13
4.2. Especificación Funcional . . . 14
4.3. Restricciones . . . 16
4.3.1. Restricciones Generales . . . 16
4.3.2. Restricciones de *Software* . . . 16
4.3.3. Restricciones de *Hardware* . . . 17

5. DISEÑO DE LA SOLUCIÓN **18**
5.1. Arquitectura . . . 18

5.2. Vista de arquitectura 23
5.2.1. Vista lógica 23
5.2.2. Vista Física 24
5.2.3. Vista de procesos 25
5.3. Diseño detallado 25
5.3.1. Estructura del sistema 25
5.3.2. Análisis de señales 31
5.3.3. Interfaz de usuario (UI/UX) 39
5.4. Tecnologías 45

6. DESARROLLO DE LA SOLUCIÓN 46
6.1. Implementación 46
6.2. Producto 46
6.2.1. Funcionalidad 1 46
6.2.2. Funcionalidad 2 46
6.2.3. Funcionalidad 3 46

7. PRUEBAS DE DESARROLLO 47
7.1. Metodología aplicada 47
7.2. Entorno y herramientas 47
7.3. Organización y documentación 47

8. RESULTADOS 48
8.1. Resumen de pruebas unitarias 48
8.2. Observaciones y hallazgos 48
8.3. Resumen de pruebas comparativas contra el sistema de referencia 48
8.4. Observaciones y hallazgos 48
8.5. Resumen de pruebas de aceptación del usuario 48
8.6. Observaciones y hallazgos 48

9. CONCLUSIONES 49
9.1. Análisis de Impacto del Proyecto 49
9.1.1. A corto, mediano y largo plazo 49
9.1.2. Impacto en Ingeniería de Sistemas 49
9.1.3. Impacto en Contextos Globales, Económicos, Ambientales y Sociales 49
9.2. Conclusiones y Trabajo Futuro 49
9.2.1. Conclusiones 49

9.2.2. Trabajo futuro 49

10. REFERENCIAS **50**

11. ANEXOS **53**

11.1. MRCU - Matriz de Requerimientos y Casos de Uso 53
11.2. PMP — *Project Management Plan* 53
11.3. SAD - *Software Architecture Document* 53
11.4. SDD - *Software Design Document* 53
11.5. SRS - *Software Requirements Specifications* 53
11.6. TD - *Test Document* 53
11.7. MU - Manual de Usuario 53
11.8. VFP - Versión Final Propuesta de Proyecto 53
11.9. Aplicación de escritorio 53

1 INTRODUCCIÓN

[Escriba aquí el contenido de esta sección.]

2 DESCRIPCIÓN GENERAL

2.1 Oportunidad y problema

Las señales electroencefalográficas (EEG) son el registro de la actividad eléctrica generada por el cerebro, captada mediante electrodos. Esta actividad se puede representar mediante ondas cerebrales que se pueden estudiar en diferentes dominios de tiempo, frecuencia o tiempo-frecuencia, reflejando diferentes estados del individuo [1].

Una de las ventajas de estas señales es su alta resolución temporal, la cual permite observar cambios en la actividad neuronal en cuestión de milisegundos [2]. Esta característica convierte al EEG en una fuente de información valiosa para el estudio del sistema sensorial y motor, ya que permite seguir de cerca cómo el cerebro percibe su entorno y coordina el movimiento, procesos que ocurren y se transforman en fracciones de segundo [3]. Sin embargo, esta misma riqueza de información hace que las señales sean complejas: contienen una gran cantidad de datos que no son fáciles de interpretar a simple vista, por lo que se requieren herramientas capaces de organizarlas, limpiarlas y resaltar los patrones relevantes para su análisis.

En este escenario, un sistema computacional especializado en el tratamiento de estas señales permite potenciar dicho aprovechamiento, mejorando procesos de visualización, filtrado y extracción de características [4]. No obstante, alcanzar este nivel de procesamiento exige conocimientos avanzados de programación o el manejo de plataformas especializadas, una barrera técnica ampliamente documentada incluso en herramientas de código abierto orientadas a facilitar el trabajo del investigador [5]. Es en este contexto donde se enmarca Gamma Lab, una solución existente que, por tratarse de una primera iteración, presenta oportunidades de mejora en cuanto a funcionalidad y usabilidad. Sobre esta base, el presente trabajo desarrolla **la segunda versión** de dicha herramienta, ampliando sus funciones y corrigiendo las limitaciones identificadas en la primera iteración.

2.1.1 Contexto del Problema

GammaLab es una plataforma de *software* de código abierto creada en 2025. Su desarrollo respondió a una necesidad del Laboratorio de Electrofisiología de la Universidad Nacional de Colombia, cuyo flujo de trabajo dependía de herramientas fragmentadas y de licenciamiento comercial, lo que limitaba la reproducibilidad y el acceso de investigadores sin formación en programación. Frente a esto, se desarrolló un *software* libre, con arquitectura *microkernel* y ecosistema de *plugins* en Python, que permitiera el análisis de señales EEG en los dominios de tiempo, frecuencia y tiempo-frecuencia mediante una interfaz gráfica única [6].

Como se expone en la memoria de trabajo de grado [6], Gamma Lab fue validada mediante pruebas comparativas frente al sistema de referencia BOARD-FTD-PACC [7], con un 87.5 % de coincidencia en los resultados, y mediante pruebas de aceptación con el usuario final, evaluando los 47 requerimientos funcionales definidos. De estos, 37 fueron aceptados sin reservas, 4 se aceptaron parcialmente y 6 quedaron fuera del alcance implementado; entre ellos la gestión de proyectos, la persistencia de sesiones de trabajo y el soporte a formatos adicionales de señal. A esto se suman las limitaciones que los propios usuarios señalaron en el control de la densidad de muestreo y en la selección de filtros avanzados [6]. Evidenciando que, si bien la primera versión validó la viabilidad de una alternativa libre y modular a las herramientas comerciales, dejó abiertas necesidades funcionales y de robustez que motivan el desarrollo de una segunda versión.

2.1.2 Formulación del Problema

Gamma Lab 1.0 [6] representó una solución inicial para la visualización y el procesamiento de señales EEG mediante una aplicación con interfaz gráfica. Sin embargo, propio de las limitaciones de una primera versión, se identificaron diversas brechas, entendidas tanto como deficiencias en funcionalidades existentes que requieren corrección como oportunidades de mejora asociadas a funcionalidades ausentes, o presentes de manera parcial y que aún no alcanzan un nivel de madurez adecuado. Las pruebas iniciales realizadas con usuarios del Laboratorio de Electrofisiología permitieron identificar estas brechas en los siguientes cuatro frentes principales:

- **Corrección de Errores Funcionales.** Se identificaron inconsistencias en diferentes módulos de procesamiento y análisis. Estas incluyen la ejecución manual de la remoción de artefactos sin un criterio de procesamiento definido, una implementación incorrecta de la normalización de la transformada *Wavelet*, fallas en la exportación de resultados hacia formatos CSV y JSON, ausencia de exportación en formato EMF, limitaciones en el cálculo de mediciones sobre múltiples ensayos y señales procesadas, y bloqueos de la aplicación durante la ejecución de los filtros elíptico y Chebyshev.

- **Fortalecimiento de la Experiencia de Navegación e Interacción.** A pesar del conocimiento técnico de los usuarios, la interfaz no comunica con claridad el estado del sistema ni los distintos modos de interacción disponibles, dificultando la navegación. Estas limitaciones se relacionan con la ausencia de identificadores de la señal, canal o análisis activo, la falta de retroalimentación visual sobre los elementos interactivos y los modos de selección, y la ausencia de ayudas contextuales para la configuración de parámetros. En conjunto, estos aspectos dificultan la construcción de un modelo mental

claro del funcionamiento de la aplicación para el usuario.

- **Estabilización de la Arquitectura y el Rendimiento del Sistema.** La revisión del *kernel* evidenció que el componente encargado de almacenar las señales y los resultados de análisis no ejecuta ninguna rutina de liberación de memoria cuando el usuario carga una nueva señal sin cerrar la aplicación, existe una función que registra la señal activa en el *buffer* interno, pero no una función complementaria que libere los datos de la sesión anterior. En consecuencia, el estado de cada carga se acumula indefinidamente en memoria durante la ejecución del programa, lo que deriva en un consumo creciente de recursos y en el bloqueo de la aplicación.

  Asimismo, la revisión del código confirmó que las funciones de procesamiento de la gran mayoría de los módulos de análisis se ejecutan de forma síncrona sobre el mismo hilo de la interfaz gráfica, sin que el núcleo del sistema provea un mecanismo compartido para delegar estos cálculos a procesos en segundo plano. Solo en casos puntuales se han implementado soluciones aisladas para evitar que la interfaz se congele durante un cálculo extenso por medio de un hilo de ejecución propio, pero no a nivel estructural de la herramienta. Esta dependencia de un único flujo de ejecución principal genera un cuello de botella que se agrava a medida que aumenta el tamaño de las señales o la complejidad de los análisis solicitados.

- **Ampliación del Alcance Analítico.** Las funcionalidades actuales de la herramienta se concentran en describir la amplitud, la distribución de energía por banda y la evolución temporal de la potencia de una misma oscilación. Sin embargo, estas técnicas no permiten caracterizar la relación entre distintos ritmos cerebrales que ocurren de forma simultánea sobre una misma señal. Esto representa una oportunidad para incorporar métodos de análisis complementarios orientados a cuantificar el acoplamiento entre componentes de distinta frecuencia de una misma señal, ampliando así la profundidad interpretativa que Gamma Lab puede ofrecer.

En conjunto, las brechas identificadas evidencian que la herramienta, aún no reúne las condiciones necesarias para consolidarse como una plataforma de uso sostenido dentro del Laboratorio de Electrofisiología. Esto plantea la necesidad de una segunda iteración orientada a fortalecer la plataforma sobre una base técnica que permita su evolución, adopción y utilización en el tiempo.

2.1.3 Propuesta de Solución

En respuesta a lo anterior, se plantea el desarrollo de Gamma Lab 2.0 como la segunda iteración de la plataforma, concebida para consolidar la base técnica establecida en la primera

versión y dar continuidad a su evolución como herramienta para el procesamiento y análisis de señales EEG. La solución se organiza en torno a los mismos cuatro frentes descritos en la formulación del problema, de modo que cada brecha identificada cuenta con una medida concreta que la resuelve.

- **Corrección de Errores Funcionales en los Análisis y Procesos Existentes.** Este frente comprende la actualización de los módulos de procesamiento y análisis para garantizar un comportamiento consistente durante la ejecución de la herramienta. La propuesta incorpora la revisión del procedimiento de remoción de artefactos, la corrección de la normalización de la transformada *Wavelet*, la actualización de los mecanismos de exportación de resultados, la ampliación del módulo de mediciones para soportar múltiples ensayos y señales procesadas, la incorporación del formato de exportación EMF y la estabilización de los filtros digitales implementados.

- **Fortalecimiento de la Experiencia de Navegación e Interacción.** La solución redefine la interacción con la herramienta mediante una interfaz que comunica de forma explícita el contexto de trabajo del usuario. Para ello, incorpora elementos de contextualización de la señal, el canal y el análisis activo, indicadores visuales del estado de interacción, ayudas para la configuración de parámetros y una reorganización de la navegación que reduzca la dependencia de menús contextuales como mecanismo principal de operación. Adicionalmente, incorpora la persistencia de proyectos y sesiones de trabajo.

- **Estabilización de la Arquitectura y el Rendimiento del Sistema.** La propuesta fortalece el núcleo de la aplicación mediante mecanismos para la gestión del ciclo de vida de los recursos utilizados durante el procesamiento de señales y la revisión del esquema de ejecución de los módulos de análisis. Como parte de esta evolución, se plantea la incorporación de un mecanismo compartido de ejecución en segundo plano dentro del *kernel* como orquestador, que permita desacoplar el procesamiento de la interfaz gráfica y establecer una base común para soportar análisis de mayor demanda computacional.

- **Ampliación del Alcance Analítico de la Herramienta.** La segunda iteración incorpora un nuevo módulo para el análisis de *Phase-Amplitude Coupling* (PAC), ampliando las capacidades hacia el estudio de la interacción entre componentes de distinta frecuencia de una misma señal. El módulo se integra bajo la arquitectura de *plugins* de la plataforma, permitiendo su incorporación como una capacidad analítica adicional dentro del flujo de procesamiento existente.

2.1.4 Justificación de la solución

La extensión de Gamma Lab busca consolidarla como una herramienta capaz de mantenerse y evolucionar en el tiempo. Las brechas identificadas afectan atributos de calidad del *software*, entre ellos la estabilidad, la extensibilidad, la usabilidad y la confiabilidad, de los cuales depende la adopción de esta como plataforma de investigación dentro del laboratorio. En particular, una arquitectura inestable compromete la escalabilidad prevista por el diseño de *microkernel* e impide extender la herramienta de forma consistente; una experiencia de navegación poco intuitiva reduce la accesibilidad que motivó el desarrollo de la interfaz gráfica como alternativa a herramientas basadas en programación; y la presencia de errores funcionales compromete la confiabilidad de los resultados obtenidos.

Además, es importante considerar que Gamma Lab está concebido para operar en un entorno con baja disponibilidad de mantenimiento por parte del equipo desarrollador, por lo que su utilización no puede depender de soporte continuo para interpretar la interfaz o resolver incidencias durante la ejecución. En estas condiciones, la herramienta debe ofrecer una interacción autónoma, resultados analíticos verificables y una infraestructura de *software* capaz de soportar su evolución de manera estable. Solo sobre esta base resulta pertinente ampliar su alcance analítico, permitiendo consolidar a Gamma Lab como una plataforma de *software* libre, sostenible y extensible para el procesamiento y análisis de señales EEG.

2.2 Descripción del Proyecto

Gamma Lab 2.0 corresponde a la segunda iteración de la plataforma de *software* libre para el procesamiento y análisis de señales EEG utilizada por el Laboratorio de Electrofisiología de la Universidad Nacional de Colombia. El proyecto abarca la corrección de las funcionalidades existentes, el fortalecimiento de la base técnica y arquitectónica de la herramienta, y la ampliación de su nivel de análisis, partiendo de la arquitectura *microkernel* y del ecosistema de *plugins* ya establecidos en la primera versión, y conservando su enfoque de interfaz gráfica como alternativa accesible frente a herramientas comerciales o basadas en programación. Con el propósito de transformar a Gamma Lab de un prototipo funcional en una plataforma consolidada, capaz de responder con solidez a las exigencias de la investigación en electrofisiología.

2.2.1 Objetivo General

Desarrollar GammaLab 2.0 como una versión extendida de la aplicación original que amplíe las capacidades de análisis de señales EEG e incorpore una arquitectura escalable, con el fin de mejorar el rendimiento del sistema y soportar un mayor número de análisis en los

dominios de tiempo, frecuencia y tiempo-frecuencia.

2.2.2 Objetivos Específicos

1. Identificar brechas de mejora en la versión 1.0 Gamma Lab.
2. Diseñar el planteamiento estructural de la solución para abordar brechas detectadas.
3. Implementar la solución a través de *software*.
4. Probar la solución mediante validaciones del usuario.

2.3 Entregables, estándares utilizados y justificación

[Escriba aquí el contenido de esta sección.]

3 CONTEXTO DEL PROYECTO

3.1 Trasfondo

Gamma Lab 2.0 busca ampliar y estabilizar una herramienta que ya opera sobre un área concreta: la actividad eléctrica del cerebro registrada mediante *EEG*. Para comprender el alcance de esta ampliación, es necesario partir de la naturaleza de la señal que se analiza. Esta actividad es la manifestación macroscópica de la comunicación entre neuronas, que se transmiten información mediante impulsos eléctricos [13]. Cuando estos impulsos se sincronizan a gran escala, se organizan en oscilaciones que un EEG registra y que la comunidad científica clasifica en bandas de frecuencia estandarizadas [14]: Delta (0,5-4 Hz) y Theta (4-8 Hz), predominantes en estados de sueño profundo o de conciencia reducida [15]; Alpha (8-13 Hz), asociada a la relajación en vigilia [14]; Beta (13-30 Hz), vinculada a la atención activa [14]; y Gamma (mayor a 30 Hz), asociada a procesos de integración neuronal y cómputo cognitivo complejo [16]. Estas oscilaciones constituyen, por tanto, uno de los principales elementos sobre los que se estructura el análisis de señales EEG y, en particular, el procesamiento realizado por la herramienta.

Sin embargo, para estudiar estas oscilaciones no basta con analizar la señal desde una única perspectiva. Una misma señal EEG puede estudiarse desde diferentes dominios de análisis, en donde cada uno permite caracterizar un aspecto particular de su comportamiento:

- **Dominio del tiempo**: caracteriza la representación de la señal respecto al tiempo. Examina parámetros como su amplitud, duración y morfología, e incluye los Potenciales Relacionados con Eventos (ERP), respuestas cerebrales promediadas y sincronizadas con la ocurrencia de un estímulo [2].

- **Dominio de la frecuencia**: caracteriza la composición espectral de la señal. Mediante la Transformada de Fourier (*FFT*) descompone la señal en sus componentes de frecuencia, y mediante la Densidad Espectral de Potencia (*PSD*) cuantifica la contribución de cada banda de frecuencia [1, 4].

- **Dominio de tiempo-frecuencia**: caracteriza cómo evoluciona la composición espectral de la señal a lo largo del tiempo. Combina ambas perspectivas mediante la Transformada de Fourier de Tiempo Corto (*STFT*), que segmenta la señal en ventanas temporales sucesivas, o la Transformada de *Wavelet* (*WT*), que examina la señal en múltiples escalas simultáneamente, proporcionando mayor resolución temporal en las frecuencias altas y mayor resolución espectral en las bajas [4].

Estos tres dominios son, en conjunto, el lenguaje técnico sobre el que ya opera Gamma Lab y sobre el que se apoyan sus módulos de preprocesamiento, *wavelet* y promediado de ensayos.

Ahora bien, estos tres dominios comparten una limitación de fondo: analizan las bandas de oscilación de manera individual. Por ello, no permiten caracterizar directamente qué ocurre cuando dos bandas distintas se activan de forma simultánea sobre una misma señal y existe una relación entre ellas. Este tipo de interacción se conoce como acoplamiento cruzado de frecuencias (*Cross-Frequency Coupling*, CFC). Dentro de sus distintas formas, el acoplamiento fase-amplitud (*Phase-Amplitude Coupling*, PAC) describe una relación en la que la fase de una oscilación lenta modula la amplitud de una oscilación rápida; este fenómeno se ha asociado con procesos de comunicación, coordinación y cómputo neuronal a través de distintas escalas espaciales y temporales [17]. Su análisis requiere, por tanto, medidas específicas de acoplamiento, como el índice de modulación para relaciones dentro de un mismo canal [17] y la Consistencia de Fase por Pares (*Pairwise Phase Consistency*, PPC) para relaciones entre canales [11]. Estas medidas constituyen la base del nuevo módulo PAC que Gamma Lab 2.0 incorpora, tanto para el análisis de un solo canal como para el análisis inter-canal.

La incorporación de esta nueva capacidad analítica, sin reescribir lo que ya existe, se apoya en una decisión arquitectónica adoptada desde la primera versión de Gamma Lab. La herramienta se construyó sobre una arquitectura de *microkernel*, un patrón de diseño en el que un núcleo central concentra los servicios esenciales del sistema, mientras que cada dominio o técnica de análisis se implementa como un *plugin* independiente que puede incorporarse sin modificar el núcleo ni los demás módulos [18]. Esta estructura favorece la extensibilidad del sistema y permite incorporar nuevas técnicas de análisis, como PAC. Sin embargo, el crecimiento de los módulos también incrementa las exigencias sobre el núcleo, especialmente en la coordinación y gestión de los procesos de cómputo [18]. En su estado actual, la arquitectura se encuentra próxima a su límite de capacidad para sostener de manera estable las funcionalidades que ya integra, por lo que la incorporación de nuevas capacidades hace necesario fortalecer los mecanismos de coordinación y evolucionar el núcleo hacia un mecanismo de orquestación que permita gestionar de forma centralizada y eficiente el procesamiento de los distintos módulos de análisis.

En conjunto, estos elementos delimitan el trasfondo técnico y analítico de Gamma Lab 2.0. La ampliación hacia PAC y la estabilización del núcleo son los frentes que exigen esta fundamentación conceptual previa; la corrección de errores funcionales y el fortalecimiento de la experiencia de interacción completan el alcance de esta segunda iteración. A partir de esta base, es posible situar la propuesta frente a las soluciones existentes y reconocer las aproximaciones que actualmente se emplean para el análisis de señales EEG.

3.2 Alternativas de solución

A continuación, se presentan cuatro herramientas ya existentes para el análisis y procesamiento de señales EEG, seleccionadas por abordar problemáticas similares a las que enfrenta Gamma Lab 2.0. El objetivo es identificar sus capacidades y limitaciones frente al alcance definido para esta segunda iteración, retomando el ejercicio comparativo realizado durante el desarrollo de Gamma Lab 1.0 [6].

3.2.1 Alternativa 1

EEGLAB es una caja de herramientas interactiva de MATLAB para el procesamiento de datos electrofisiológicos continuos y relacionados con eventos (EEG, MEG y otras modalidades), que incorpora análisis de componentes independientes (ICA), análisis en los dominios de tiempo y frecuencia, rechazo de artefactos, estadísticas relacionadas con eventos y distintos modos de visualización de datos promediados y de ensayos individuales [19]. Al operar sobre MATLAB, su uso requiere la licencia de este *software*, lo que limita su adopción en laboratorios con presupuesto restringido, además de exigir conocimientos avanzados de programación.

3.2.2 Alternativa 2

FieldTrip es una caja de herramientas desarrollada en MATLAB que ofrece un conjunto de funciones de alto nivel para el análisis de datos neurofisiológicos. A diferencia de otros entornos, no dispone de interfaz gráfica de usuario: las funciones se combinan dentro de un flujo de trabajo programado mediante un *script* en MATLAB que integra todas las etapas del análisis [20]. Esta característica hace que la herramienta resulte poco intuitiva para quienes se inician en la programación en MATLAB o en el análisis de señales EEG, MEG, iEEG o NIRS, a lo que se suma la misma barrera de licenciamiento que presenta EEGLAB.

3.2.3 Alternativa 3

Brainstorm es una aplicación colaborativa y de código abierto dedicada al análisis de grabaciones cerebrales (MEG, EEG, fNIRS, ECoG, electrodos de profundidad y electrofisiología multiunidad). Su uso es gratuito y no requiere licencia de MATLAB [21]. A través de su interfaz gráfica permite cubrir las necesidades de análisis sin necesidad de programar; sin embargo, dicha interfaz resulta poco intuitiva y su baja personalización introduce cierta rigidez en el flujo de trabajo.

3.2.4 Alternativa 4

BOARD-FTD-PACC es una interfaz gráfica de usuario desarrollada en MATLAB, diseñada para la visualización, cuantificación y análisis de registros neurofisiológicos. Proporciona herramientas personalizables orientadas al análisis de la actividad postsináptica y de datos oscilatorios neuronales complejos, principalmente mediante análisis cruzado de frecuencias [7]. Al ser el antecedente directo sobre el que se validó Gamma Lab 1.0, constituyó la base conceptual del proyecto; no obstante, su dependencia de una licencia de MATLAB y su interfaz poco intuitiva restringen su adopción en entornos con recursos limitados.

3.3 Comparación de alternativas

A continuación, se presenta una tabla comparativa que resume las características clave de las alternativas. Evaluando licenciamiento, costo, interfaz gráfica, independencia de MATLAB y conocimientos de programación requeridos; factores que determinan la accesibilidad y la viabilidad de adopción en un entorno de investigación con recursos limitados como el del Laboratorio de Electrofisiología.

Criterio	**EEGLAB**	**FieldTrip**	**Brainstorm**	**BOARD-FTD-PACC**	**GAMMA LAB 2.0**
Licencia gratuita	Parcial	Parcial	Parcial	Parcial	Sí
Costo anual (COP)	$3.948.000	$3.948.000	$0	$3.948.000	$0
GUI integrada	Parcial	No	Sí	Parcial	Sí
Independencia MATLAB	Si	Si	No	Si	No
Conocimientos en programación	Alto	Bajo	Alto	Bajo	Bajo

Tabla 1: Comparación de alternativas para análisis de señales neurofisiológicas

Como se evidencia en la tabla 1, Gamma Lab 2.0 consolida las ventajas de su versión anterior e incorpora mejoras clave frente a las alternativas existentes. Es la única alternativa completamente gratuita que además no requiere conocimientos avanzados de programación, eliminando así las dos barreras de acceso más comunes en laboratorios con recursos limitados. Gamma Lab 2.0 combina accesibilidad económica con una interfaz gráfica intuitiva, reduciendo significativamente la curva de aprendizaje para nuevos investigadores. BOARD-FTD-PACC, al ser el predecesor directo de Gamma Lab, sirvió como base conceptual del proyecto, pero requiere licencia de MATLAB, lo que limita su adopción en entornos con presupuestos restringidos, además de contar con una interfaz poco intuitiva que dificulta su uso para personas sin conocimientos en programación. Gamma Lab 2.0 aporta así al ecosistema de ciencia abierta una alternativa libre, accesible y personalizable para el análisis de señales neurofisiológicas, eliminando la barrera de los conocimientos en programación sin comprometer la profundidad analítica del *software*.

4 ANÁLISIS DEL PROBLEMA

4.1 Requerimientos

El levantamiento de requerimientos para Gamma Lab 2.0 partió de las brechas identificadas en la versión antecesora y de las necesidades expresadas por el Laboratorio de Electrofisiología. El conjunto resultante de este proceso se documentó bajo el estándar ISO/IEC/IEEE 29148 [22] y se clasificó en funcionales y no funcionales. En total se identificaron **81 requerimientos**, de los cuales **45 son funcionales y 36 no funcionales**. Las siguientes subsecciones presentan una explicación general de ambos grupos y los criterios bajo los cuales se organizaron; el detalle uno a uno se documenta en el anexo de Especificación de Requisitos de *Software* (SRS) (FALTA CITAR ANEXO AQUÍ) , y la información presentada a continuación se alinea con dicho documento.

4.1.1 Requerimientos Funcionales

Los requisitos funcionales especifican las acciones y comportamientos observables que Gamma Lab 2.0 debe ejecutar ante una entrada, evento o solicitud del usuario. Para facilitar su organización y trazabilidad, estos se agruparon en módulos de acuerdo con la capacidad que habilitan, considerando aspectos como el tipo de dato que manejan, la operación que realizan sobre la señal y el propósito que cumplen dentro del flujo de trabajo. De este proceso surgieron cuatro módulos: IO y UI, de carácter transversal y presentes a lo largo de los flujos de análisis, y WV y PAC, de carácter específico y vinculados a técnicas de análisis particulares. Esta organización refleja, a su vez, los frentes de trabajo definidos para la solución: bajo esta lógica, IO y UI buscan fortalecer la funcionalidad base y mejorar la experiencia de usuario; WV se enfoca en corregir y fortalecer el funcionamiento de una técnica ya incorporada en Gamma Lab; mientras que PAC amplía el alcance analítico de la herramienta mediante la incorporación de una nueva técnica de análisis.

- **Módulo IO – Entrada/Salida.** Opera sobre los datos que entran y salen de la herramienta: la señal en sus formatos de origen (ABF, EDF, MAT), el proyecto de trabajo (*.glab*) y los resultados de los análisis. Sus operaciones son de carga, persistencia y exportación. En esa medida, IO garantiza la persistencia de los resultados y del estado del proyecto, permitiendo al usuario retomar su trabajo sin pérdida de información y disponer de sus resultados fuera de la herramienta.

- **Módulo UI – Interacción general.** Opera sobre la señal ya cargada y los *trials* derivados de ella, es decir, sobre los datos con los que el usuario interactúa directamente. Sus

operaciones son de edición y navegación: recortar, ajustar el muestreo, remover artefactos, hacer *zoom* y seleccionar o promediar *trials*. En esa medida, UI otorga al usuario control preciso y visible sobre la señal, mejorando su experiencia de interacción con las capacidades ya existentes de la herramienta.

- **Módulo WV – *Wavelet*/Escalograma.** Opera sobre el resultado de una técnica ya presente en GammaLab: el escalograma generado por *wavelet*. Sus operaciones son de ajuste y presentación, entre las que se incluyen elegir la escala, normalizar el resultado y exportarlo en formato vectorial, sin intervenir en el cálculo del escalograma en sí. En esa medida, WV corrige y fortalece esta técnica existente, dándole al usuario mayor control sobre su interpretación visual.

- **Módulo PAC – Acoplamiento fase-amplitud.** Opera sobre una relación nueva entre los datos de la señal: el acoplamiento entre la fase de una banda lenta y la amplitud de una banda rápida, intra e inter-canal. Sus operaciones cubren todo el ciclo del análisis, desde la selección de bandas y canales hasta el cálculo del acoplamiento, la coherencia de fase y la exportación de resultados. En esa medida, PAC amplía el alcance analítico de GammaLab mediante una técnica completamente nueva, concentrando cerca de la mitad de los requisitos funcionales de esta iteración.

4.1.2 Requerimientos No Funcionales

Los requisitos no funcionales definen las condiciones de calidad que debe cumplir Gamma Lab 2.0 durante su operación. Para su clasificación se utiliza el modelo FURPS+ [23], que contempla las categorías de usabilidad (U), rendimiento (P), confiabilidad (R) y soporte (S). En esta iteración, estos requisitos se enfocan principalmente en estabilizar la arquitectura y mejorar el rendimiento del sistema, mediante la consolidación del núcleo y el orquestador de tareas. Además, complementan los objetivos orientados a la corrección de errores y al mejoramiento de la experiencia de usuario.

- **U – Usabilidad.** Agrupa los requisitos orientados a mejorar la experiencia de interacción del usuario con la herramienta: retroalimentación visual durante operaciones prolongadas, diferenciación de mensajes según su severidad, y accesibilidad de las funcionalidades desde la interfaz principal.

- **P – Rendimiento.** Establece los tiempos máximos de respuesta y límites de consumo de recursos para las operaciones del sistema: carga de señales, cómputo de análisis, exportación de resultados y liberación de memoria tras el cierre de un proyecto o módulo.

- **R – Confiabilidad.** Garantiza la integridad y consistencia del sistema durante su operación: sincronización del acceso a datos compartidos, liberación de recursos al finalizar procesos, y preservación de la señal y los *trials* originales frente a cualquier operación de preprocesamiento.

- **S – Soporte.** Incluye los requisitos que sostienen la arquitectura interna de la herramienta: el registro centralizado de servicios en el núcleo, la ejecución de cómputo pesado a través del orquestador de tareas, y la incorporación de nuevos *plugins* de análisis sin modificar los componentes existentes.

4.2 Especificación Funcional

A partir del levantamiento de requerimientos, se construyeron los casos de uso del sistema, tomando cada requisito como insumo para identificar el flujo de trabajo, el actor involucrado y la funcionalidad que respondía a la necesidad correspondiente. Los requerimientos afines en flujo y propósito se integraron en un mismo caso de uso, mientras que las interacciones independientes dieron lugar a casos de uso propios. De este proceso resultaron **23 casos de uso, cada uno con al menos un requisito asociado**, garantizando la cobertura total de los requerimientos; la asignación entre requisitos y casos de uso que se documenta en el anexo de especificación de casos de uso. (FALTA CITAR ANEXO AQUÍ) .

A partir de los casos de uso definidos se configura una visión integral del sistema, en la que las funcionalidades se articulan para representar las capacidades de Gamma Lab 2.0 y las relaciones entre los componentes que intervienen en su ejecución. Como se observa en el diagrama 1, el modelo contempla un único actor, el Investigador, quien desencadena las funcionalidades del sistema mediante sus solicitudes, sin que se representen actores externos o procesos autónomos como iniciadores de casos de uso. La interacción se concentra inicialmente en las funcionalidades de Interfaz y Usabilidad, desde las cuales se accede a las capacidades de la herramienta y se establece la comunicación con el Núcleo y Ciclo de Vida, que actúa como elemento central para gestionar el estado de la aplicación y coordinar la ejecución de los procesos. A partir de este núcleo se integran los subsistemas de Preprocesamiento, Análisis y Exportación, que soportan las diferentes etapas del flujo de trabajo con las señales. Esta lectura por componentes es la que guía la construcción de Gamma Lab 2.0: cada subsistema del diagrama 1 corresponde a un componente identificable dentro de la arquitectura del sistema.

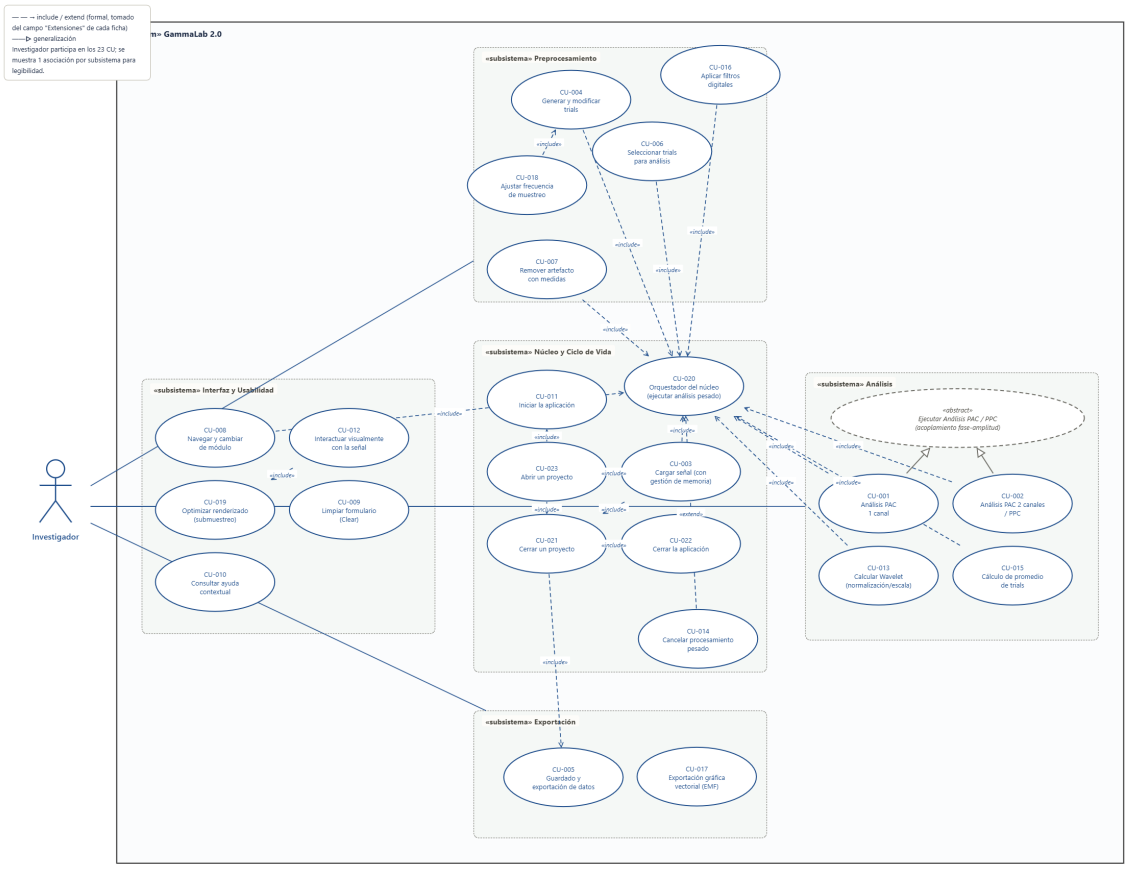


Figura 1: Diagrama de casos de uso del sistema Gamma Lab 2.0

Esta correspondencia entre subsistemas y componentes se detalla a continuación:

- **Subsistema de Núcleo y Ciclo de Vida.** Agrupa los casos de uso que gestionan el estado global de la aplicación: inicio, apertura y cierre de proyectos, carga de señal con gestión de memoria y cierre de la aplicación; junto con el orquestador del núcleo, encargado de despachar todo cómputo pesado y de habilitar su cancelación. Los casos de uso de Preprocesamiento y Análisis que implican cómputo intensivo se apoyan en este subsistema mediante relaciones de inclusión.

- **Subsistema de Interfaz y Usabilidad.** Agrupa los casos de uso que gobiernan la navegación entre módulos, la interacción visual con la señal (*zoom*, lectura de coordenadas, *butterfly*), el renderizado fluido de señales pesadas mediante submuestreo, la limpieza de formularios y la ayuda contextual.

- **Subsistema de Preprocesamiento.** Agrupa los casos de uso que transforman la señal antes de cualquier análisis: la generación y modificación de *trials*, la selección de

los *trials* que participan en un cálculo, la remoción de artefactos de estimulación con medidas de referencia previas al cambio, el ajuste de la frecuencia de muestreo y la aplicación de filtros digitales.

- **Subsistema de Análisis.** Agrupa las técnicas de procesamiento que constituyen el propósito analítico de la herramienta: el cálculo de la transformada *Wavelet*, el promedio de *trials*, y el análisis de acoplamiento fase-amplitud en su variante intra-canal (un canal) e inter-canal (dos canales y coherencia de fase, PPC), estas dos últimas modeladas como especializaciones de un caso de uso abstracto que comparte el procedimiento de cálculo.
- **Subsistema de Exportación.** Agrupa el guardado y exportación de datos del proyecto y la exportación gráfica en formato vectorial de los resultados analíticos.

4.3 Restricciones

Esta sección define los límites técnicos, normativos y operativos que condicionan el diseño, la implementación y el despliegue de Gamma Lab 2.0.

4.3.1 Restricciones Generales

El sistema opera íntegramente en inglés, tanto en la interfaz como en los mensajes de error y las alertas; ante fallos internos o errores del usuario, estos mensajes deben ser claros y no interrumpir el flujo de ejecución en curso. Asimismo, debe soportar la carga de archivos con múltiples canales y permitir ejecutar análisis sobre al menos uno de ellos, habilitando la selección de dos canales cuando el análisis sea de tipo inter-canal. Las gráficas, tablas y resultados numéricos que se generen deben poder exportarse desde la misma interfaz.

4.3.2 Restricciones de *Software*

El sistema mantiene una arquitectura modular orientada a *plugins*, en la que el *Kernel* orquesta la ejecución y permite incorporar nuevas capacidades sin modificar los *plugins* existentes. Se desarrolla en Python 3.11–3.12 y se distribuye como aplicación ejecutable localmente en Windows, aunque también puede ejecutarse directamente desde el código fuente del repositorio oficial.

El procesamiento de señales se apoya en NumPy, SciPy, MNE, TensorPAC y PyWavelets, mientras que la interfaz gráfica se construye con PySide6 y los componentes Qt; los formatos de entrada soportados son *.abf*, *.edf*, *.bdf* y *.mat*. La paralelización del *Kernel*, por su parte, se implementa mediante los mecanismos de hilos y *multiprocessing* incluidos en la biblioteca estándar de Python.

4.3.3 Restricciones de *Hardware*

El sistema debe ejecutarse en Windows 10 de 64 bits o superior, sobre un equipo que cumpla con los siguientes requerimientos mínimos:

- CPU de 4 núcleos a 2.0 GHz.
- 8 GB de RAM.
- 1 GB de VRAM.
- 3 GB de almacenamiento libre.

5 DISEÑO DE LA SOLUCIÓN

5.1 Arquitectura

Gamma Lab 2.0 toma como punto de partida la arquitectura de Gamma Lab 1.0 [6], conservando el estilo de *microkernel* y su implementación en Python. Esta decisión respondió, en parte, a que un cambio de estilo o de lenguaje habría obligado a reimplementar los dieciséis *plugins* existentes y a repetir su validación numérica, un costo que no se justificaba frente a los problemas identificados. Al mismo tiempo, conservar esta base permitió mantener la extensibilidad del sistema como mecanismo para incorporar nuevas técnicas de análisis. En consecuencia, la propuesta se orientó a fortalecer la arquitectura existente mediante la incorporación de mecanismos que permitieran mejorar la ejecución, la gestión de datos y la organización de los componentes, sin modificar el modelo general de extensibilidad.

La arquitectura de partida se organizaba alrededor de un componente *Kernel*, responsable de mantener el registro de servicios y *plugins*, y de distribuir eventos entre los módulos mediante señales de Qt. La carga dinámica de los *plugins* se realizaba a través de un gestor que identificaba sus archivos *properties.yml*, mientras que la interfaz *IPlugin* establecía las operaciones básicas para su integración con el núcleo y la interfaz gráfica. Sobre esta estructura se encontraban dieciséis *plugins*, organizados en funcionalidades de entrada y salida, preprocesamiento, medición y análisis en los dominios del tiempo, la frecuencia y el tiempo-frecuencia. La visualización se apoyaba en VTK, integrado en la interfaz desarrollada con PyQt5.

El diseño de Gamma Lab 2.0 consideró cinco necesidades arquitectónicas que orientaron la definición de la solución:

- La ejecución de los análisis debía independizarse del hilo principal de la interfaz. Para ello, se planteó un mecanismo centralizado de ejecución en segundo plano, con distribución del trabajo entre varios hilos y gestión del progreso, los errores y la cancelación de las tareas.

- La gestión de los datos debía garantizar un acceso controlado y una administración explícita de los recursos. En respuesta, se definió un esquema de almacenamiento compartido que contempla la sincronización de accesos, la propiedad de los datos y la liberación de memoria al finalizar su uso.

- Los servicios utilizados por distintos *plugins* debían mantener un estado común y evitar instancias independientes. Por esta razón, se estableció su centralización en el núcleo,

de modo que los diferentes módulos pudieran acceder a los mismos recursos y configuraciones.

- La lógica de cálculo debía separarse de los elementos encargados de la interfaz y la visualización. Esta decisión llevó a reorganizar los *plugins* para que las operaciones numéricas pudieran ejecutarse y validarse de manera independiente de su representación gráfica.
- La aplicación debía permitir conservar el contexto de trabajo y facilitar su ejecución en distintos entornos. Para ello, se incorporó el concepto de proyecto, junto con mecanismos de persistencia y recuperación de sesiones, gestión de rutas y distribución de la aplicación.

Sobre estos criterios se estructuró la solución, manteniendo los componentes centrales del *microkernel* y ampliando las capacidades de los módulos existentes. La arquitectura conserva los dieciséis *plugins* originales e incorpora tres nuevos: PAC, PPC y *Modulation Index*. Además, el diseño contempla una mayor cobertura de pruebas automatizadas incluyendo pruebas de concurrencia, cancelación, persistencia, liberación de memoria, instalación y rendimiento.

La Tabla 2 presenta una comparación de las principales características arquitectónicas y técnicas de Gamma Lab 1.0 y la solución diseñada para Gamma Lab 2.0.

Característica	**Gamma Lab 1.0**	**Gamma Lab 2.0**
Estilo arquitectónico	*Microkernel* con *plugins*	*Microkernel* con *plugins* y orquestador de tareas
Modelo de ejecución	Síncrono, en el hilo de la interfaz	Asíncrono, mediante el orquestador del núcleo
Progreso y cancelación	No disponibles	Centralizados en el orquestador
Almacén de datos	Sin sincronización ni liberación de memoria	Sincronizado, con propiedad y liberación de datos
Servicios compartidos	Instancias independientes por módulo	Instancia única registrada en el núcleo
Organización de *plugins*	Cálculo y vista en la misma clase	Cálculo separado de la interfaz y la vista
Persistencia	Inexistente	Proyectos *.glab* con esquema versionado
Distribución	*Scripts* de arranque y rutas relativas	Instalador para Windows y rutas absolutas
Plugins	16	19 (PAC, PPC y *Modulation Index*)
Archivos de pruebas	14	≈ 70

Tabla 2: Comparación arquitectónica entre Gamma Lab 1.0 y Gamma Lab 2.0

Orquestador de Tareas

La incorporación del orquestador implicó una reestructuración importante de la arquitectura de Gamma Lab 2.0, debido a su relación con otros componentes del sistema. Por su

```
Gamma-Lab-2.0/
|-- app/                        interfaz grafica (VTK, hilo propio)
|   `-- view/
|
|-- core/                       NUCLEO: logica central y servicios compartidos
|   |-- kernel.py               registro de plugins y servicios
|   |-- plugins/                gestor de carga de plugins
|   |-- services/
|   |   |-- task_service.py       orquestador de tareas (nuevo)
|   |   |-- data_store.py         manejo de datos compartidos
|   |   |-- project_service.py    persistencia de proyectos (.glab)
|   |   `-- settings_service.py   configuracion de la aplicacion
|   |-- model/                  modelos de datos (senal, trial)
|   |-- filters/                filtros de senal
|   `-- utils/                  utilidades comunes
|
`-- plugins/                    PLUGINS: modulos de analisis (via orquestador)
    |-- analysis/
    |   `-- time/
    |       `-- erp/
    |           |-- erp_plugin.py        logica (IPlugin)
    |           |-- erp_plugin_ui.py     vista (independiente)
    |           |-- compute.py           calculo (validado aparte)
    |           |-- properties.yml       metadatos y registro
    |           `-- icn_erp.png          icono
    |-- faq/
    |-- io/
    |-- measure/
    `-- preprocessing/
```

Figura 2: Árbol de paquetes Gamma Lab 2.0.

carácter transversal, resulta pertinente detallar su diseño y organización. Este mecanismo se integró al núcleo mediante el servicio *TaskService*, encargado de organizar y gestionar el ciclo de vida de las tareas.

El diseño del orquestador se basa en la gestión de *tareas*, representadas por funciones de Python junto con sus argumentos. Cada tarea se caracteriza por cinco propiedades: es **pura**, pues ejecuta un cálculo y devuelve un resultado sin modificar directamente el estado compartido; es **cerrada**, pues recibe mediante parámetros todos los datos necesarios para su ejecución; es **independiente de la interfaz**, pues no depende de componentes de Qt ni de VTK; es **interrumpible**, pues puede atender solicitudes de cancelación durante su procesamiento; y es **capaz de declarar su costo**, pues proporciona información sobre sus requerimientos computacionales para ajustar la distribución de recursos.

Bajo esta estructura, un mismo *plugin* puede enviar múltiples tareas al orquestador durante una sesión, cada una asociada a una operación de cómputo particular. La comunicación se establece mediante *submit()*, que recibe la función, sus argumentos y un identificador de propietario. Cada solicitud cuenta con los elementos que presenta la Tabla 3.

La gestión de las tareas contempla cuatro estados: *queued*, cuando permanece en espera; *running*, durante su ejecución; *done*, cuando finaliza o presenta un error; y *cancelled*, cuando

Tabla 3: Elementos del contrato de solicitud del orquestador

Elemento	**Descripción y función**
id	Identificador único de la tarea, utilizado para reconocer su ejecución y descartar resultados tardíos.
owner	Identificador del *plugin* que solicita la tarea, utilizado para agrupar solicitudes y aplicar políticas de gestión.
TaskHandle	Manejador que permite consultar el progreso, recibir notificaciones de finalización, fallo o cancelación y solicitar la interrupción.
TaskContext	Contexto opcional que proporciona mecanismos para reportar el progreso y consultar solicitudes de cancelación.
task_started	Señal global que notifica el inicio de una tarea y permite actualizar la barra de progreso.

se cancela o descarta. Sobre este ciclo de vida se aplican tres mecanismos: reemplazo, que descarta las solicitudes anteriores del mismo propietario que permanecen en espera y, en la mayoría de los módulos, cancela también la tarea en ejecución; cancelación por contexto, que permite detener las tareas asociadas a un *plugin* al abandonar su módulo o cerrar un proyecto; y cancelación cooperativa, mediante la cual las tareas consultan las solicitudes de interrupción durante su procesamiento. Si una tarea no responde dentro del tiempo establecido, el servicio la desvincula de su gestión y descarta sus resultados posteriores. Los errores se notifican mediante el manejador, permitiendo continuar con las demás solicitudes.

La ejecución se organiza en dos niveles de concurrencia. En el primero, el hilo principal de Qt administra la cola y recibe las notificaciones, mientras cada tarea se ejecuta en un hilo trabajador (*QThread*) independiente. En el segundo, determinadas tareas, como el PAC promedio y el PPC, distribuyen internamente el procesamiento de los *trials* mediante un conjunto de hilos (*ThreadPoolExecutor*). Esta distribución considera los recursos de procesamiento y memoria disponibles y conserva el orden original de los resultados. La actualización del estado compartido y las operaciones de visualización VTK permanecen en el hilo principal.

La integración del orquestador con los demás componentes se establece mediante las relaciones que presenta la Tabla 4.

Tabla 4: Relación del orquestador con los demás componentes

Componente	**Relación con el orquestador**
Kernel	Registra *TaskService* como servicio del núcleo antes del descubrimiento de los *plugins*.
IPlugin	Proporciona la interfaz *run_task()* para solicitar tareas y gestionar sus resultados.
MainWindow	Recibe las notificaciones de inicio, administra la barra de progreso y coordina las cancelaciones.
DataStore	Garantiza la integridad del estado compartido mediante una política de escritor único desde el hilo principal.
ProjectService	Integra las operaciones de preparación y almacenamiento de proyectos al mecanismo de ejecución.
Plugins	Proporcionan las funciones de cálculo que se envían al orquestador desde los módulos de análisis, preprocesamiento y entrada y salida.
parallel.py	Proporciona el mecanismo de paralelismo interno utilizado por las tareas que distribuyen el procesamiento entre varios hilos.

5.2 Vista de arquitectura

5.2.1 Vista lógica

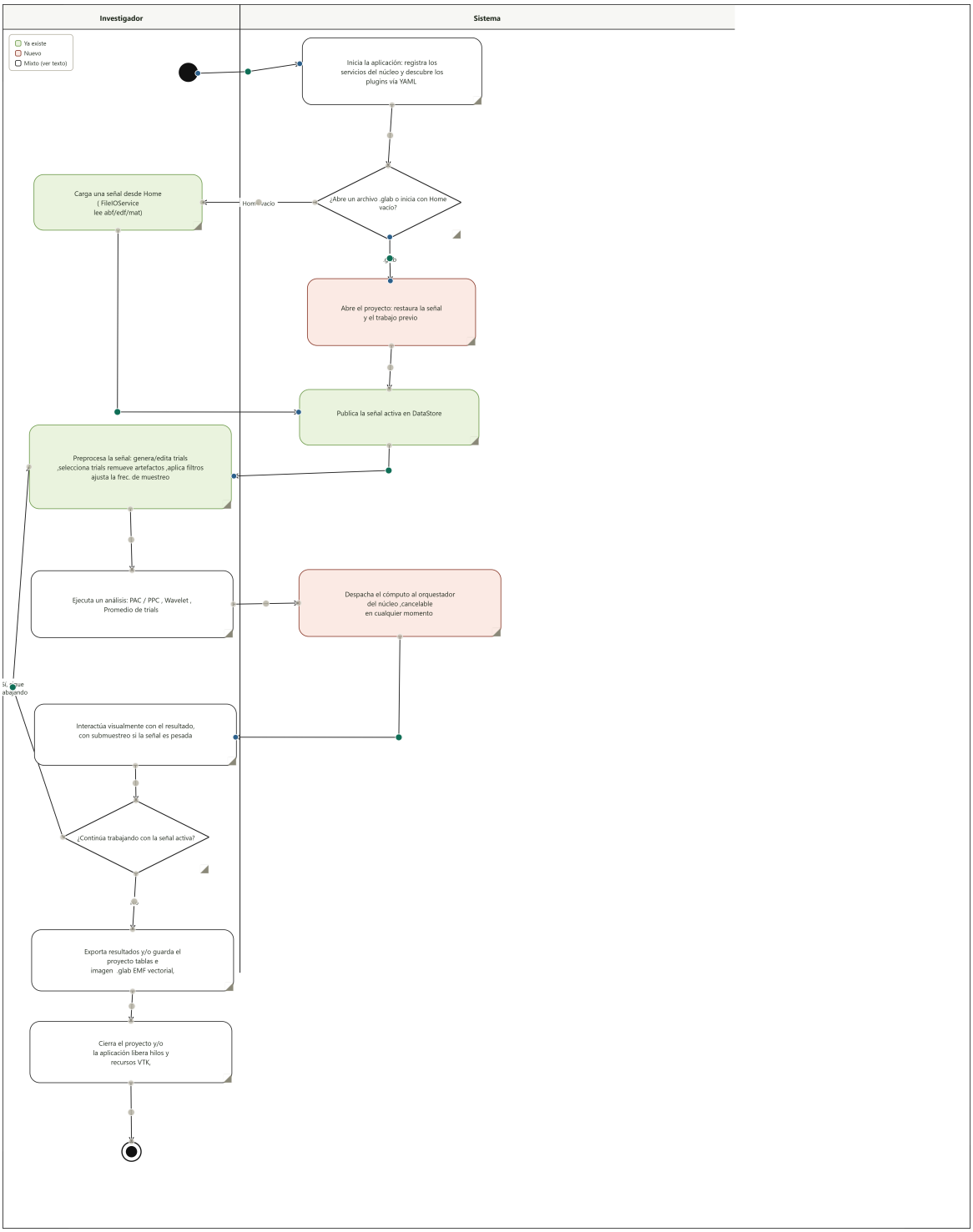


Figura 3: Diagrama de actividades de Gamma Lab 2.0

5.2.2 Vista Física

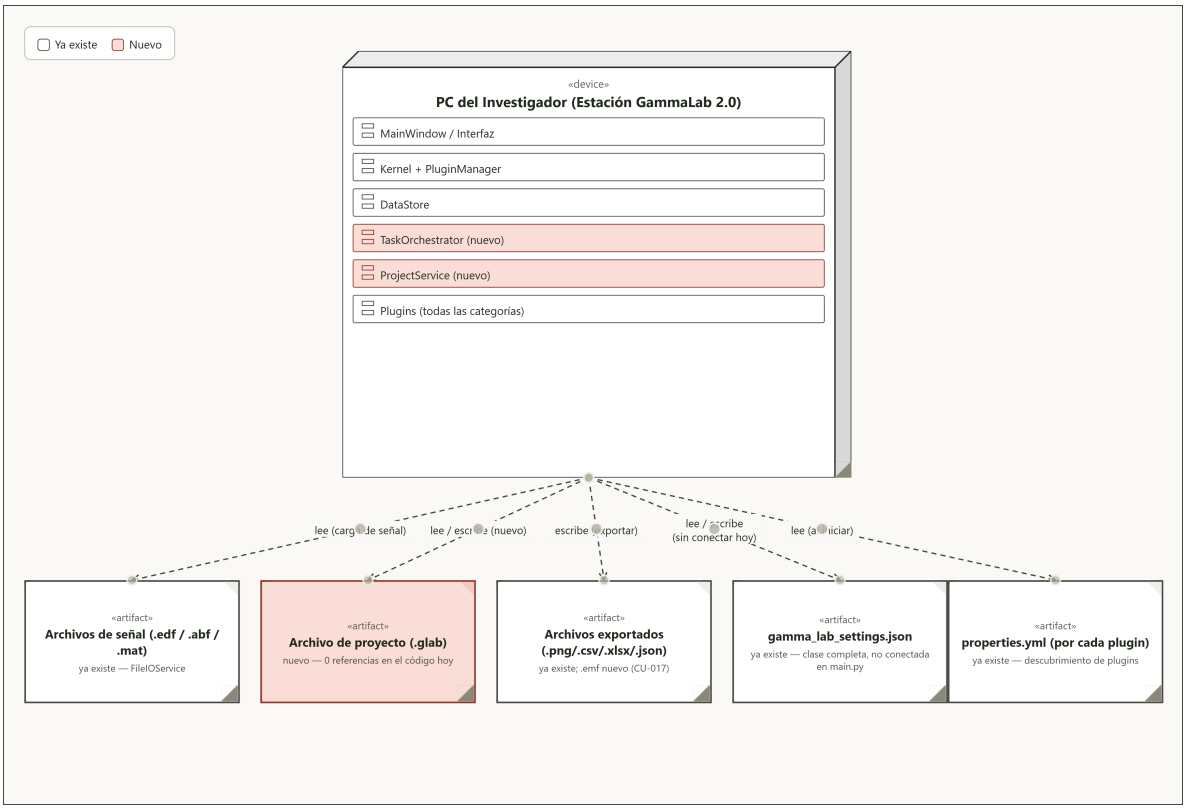


Figura 4: Diagrama de despliegue de Gamma Lab 2.0

5.2.3 Vista de procesos

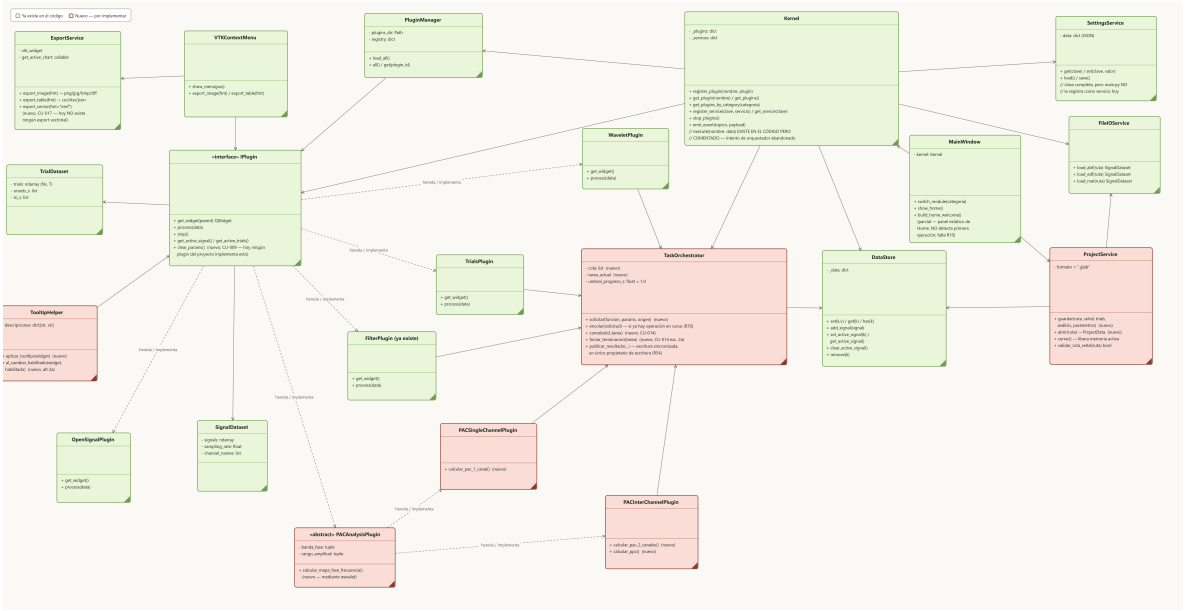


Figura 5: Diagrama de clases de Gamma Lab 2.0

5.3 Diseño detallado

5.3.1 Estructura del sistema

La Figura 6 presenta el diagrama de clases simplificado de Gamma Lab 2.0; el detalle completo se encuentra en el documento *Software Design Document* (SDD). El sistema conserva la arquitectura *microkernel* de la primera versión: el *Kernel* registra las extensiones, que implementan la interfaz *IPlugin* y son cargadas por *PluginManager*, y da acceso a los servicios que estas comparten. La evolución hacia la versión 2.0 se concentra en esos servicios. Las clases nuevas, señaladas en rojo, responden a limitaciones concretas de Gamma Lab 1.0: el orquestador de tareas, implementado como *TaskService*, resuelve la ejecución en segundo plano; *ProjectService*, la persistencia del trabajo; y las extensiones de PAC, la ampliación del alcance analítico. A continuación se explica cómo estas clases y las mejoras sobre las ya existentes se complementan, y cómo intervienen a lo largo de una sesión de trabajo.

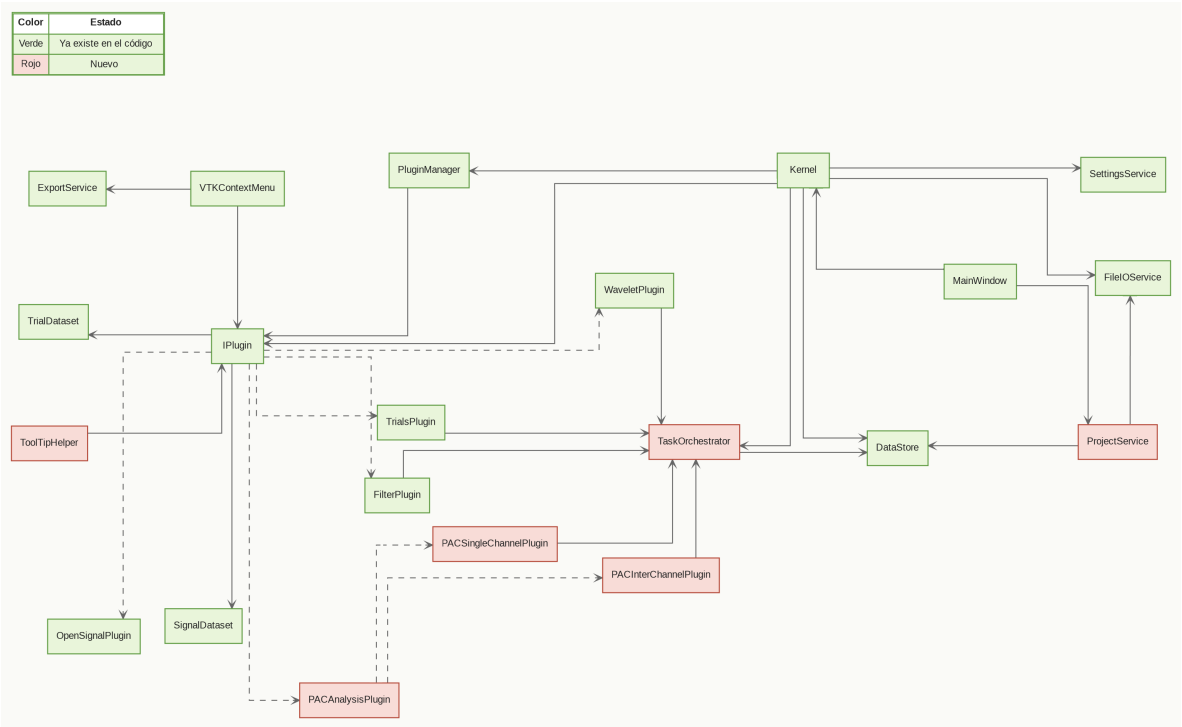


Figura 6: Diagrama de clases de Gamma Lab 2.0 simplificado

Mejoras del sistema El primer problema que debía resolverse era la estabilidad del núcleo. En Gamma Lab 1.0, el *DataStore* almacenaba la señal activa, pero no contaba con un mecanismo para liberarla, de modo que con cada nueva carga los datos anteriores permanecían en memoria y la aplicación podía bloquearse. Por ello, en Gamma Lab 2.0 se incorporó el método *release_signals()*, que al cargar una nueva señal o cerrar un proyecto elimina las señales anteriores junto con sus datos relacionados: los *trials*, los conjuntos derivados, las mediciones y el *trial* en curso. Este método forma parte de una limpieza más completa, a cargo del procedimiento *liberar_senal_anterior()*, que además cancela las tareas pendientes, cierra las ventanas de VTK y libera los recursos utilizados por los *widgets* de las extensiones.

Liberar la memoria no era suficiente, porque la nueva versión también debía ejecutar cálculos en paralelo a la interfaz, y varios hilos no pueden modificar los mismos datos sin control. Por esta razón, solo el hilo de la interfaz gráfica puede escribir en el *DataStore*; si otro hilo lo intenta, el sistema genera la excepción *EscrituraNoPermitida*. Tanto las lecturas como las escrituras utilizan además un candado reentrante, lo que reduce los posibles conflictos durante la ejecución de tareas en segundo plano.

Esta protección es la que hace posible *TaskService*. En Gamma Lab 1.0, la mayoría de los análisis se ejecutaban directamente en el hilo de la interfaz, y aunque algunas extensiones usaban sus propios hilos, no existía un mecanismo centralizado para controlarlos. En la versión

2.0, las extensiones dejaron de crear hilos: todas las tareas pasan por *TaskService*, que utiliza una única cola y procesa una tarea a la vez en un hilo de trabajo, y los resultados se envían al hilo de la interfaz antes de mostrarse. Cada solicitud devuelve un objeto *TaskHandle*, que mediante las señales *progress*, *finished*, *failed* y *cancelled* permite consultar el progreso de la tarea y saber si finalizó correctamente, presentó un error o fue cancelada.

El servicio también controla qué ocurre cuando el usuario cambia de opinión. La cancelación es cooperativa, es decir, cada tarea responde a la solicitud de cancelarse; si no lo hace en dos segundos, el servicio deja de esperarla y libera la interfaz para que el usuario pueda seguir trabajando. De forma similar, cuando una extensión agrega una nueva tarea se descartan las suyas que aún estén pendientes, de modo que presionar varias veces el botón *Generate* no genera múltiples cálculos innecesarios.

Durante el diseño de este servicio se evaluó ejecutar los cálculos en procesos independientes, pero la alternativa se descartó porque las mediciones mostraron que iniciar un proceso y enviarle los datos tomaba 1.486 ms, mientras que el cálculo en sí tardaba 1.163 ms. Se optó entonces por los hilos, ya que las operaciones más exigentes de NumPy liberan el *GIL* y permiten aprovechar varios núcleos del procesador. En las pruebas de PPC, distribuir los *trials* entre cuatro hilos permitió alcanzar una aceleración de entre 1,8× y 2,7×, manteniendo los mismos resultados.

Con la ejecución resuelta, el siguiente paso fue reducir el costo de cada cálculo. Los cálculos se realizan en formato *float64* para mantener la precisión, mientras que los datos enviados a VTK se convierten a *float32* para reducir el tamaño de lo que se visualiza. Las transformadas tiempo-frecuencia se calculan ahora por bloques de frecuencias, en lugar de procesar toda la matriz al mismo tiempo, lo que redujo el consumo de memoria de 480 MB a 30 MB para una matriz de 500×60.000. En los filtros internos de los análisis, un mecanismo de memoización guarda el diseño de los filtros IIR para no repetir cálculos, lo que redujo en 7,25 veces el tiempo necesario al cambiar de *trial*.

Sin embargo, un cálculo más rápido solo es útil si su resultado es correcto, y por eso estas mejoras se acompañaron de una revisión de los análisis existentes. Al aplicar submuestreo, el eje de frecuencias se construye ahora con la tasa de muestreo efectiva y no con la solicitada inicialmente, lo que evita desplazamientos de hasta un 11 % en dicho eje. Esa misma tasa, junto con la frecuencia de Nyquist, se tiene en cuenta al validar los parámetros *fmin* y *fmax* de la transformada *Wavelet*, cuyo método de normalización tiempo-frecuencia (*normalize_tf*) también se corrigió: ahora calcula la línea base de manera independiente para cada frecuencia y ofrece los métodos *z-score*, *percent change* y *relative power*.

En el filtrado, Gamma Lab 1.0 solo tenía implementado el filtro Butterworth, por lo que las opciones Chebyshev y elíptico fallaban al ejecutarse. La extensión de filtrado ofrece ahora

únicamente Butterworth, ejecuta el filtrado en *TaskService* y valida que el resultado no contenga valores no numéricos. Por su parte, los filtros internos de los análisis utilizan secciones de segundo orden (SOS), ya que los coeficientes en formato b/a presentan problemas de estabilidad numérica con órdenes altos. Estos filtros comprueban además que los coeficientes sean valores finitos antes de utilizarlos y devuelven copias, para evitar que otras partes del sistema modifiquen accidentalmente un filtro compartido.

La remoción de artefactos se modificó con el mismo objetivo de proteger los datos: los cambios ya no afectan directamente los *trials* originales, sino que se realizan sobre un **conjunto derivado**, creado mediante el método *crear_derivado*. Cada cambio se guarda además como una *receta*, que contiene el modo de corrección utilizado, los puntos seleccionados y los descartes realizados, lo que permite volver a aplicar el mismo procedimiento y obtener resultados reproducibles. Como se explica más adelante, esta idea de guardar cómo se obtuvo un dato, y no el dato mismo, es también la base del formato de proyecto.

Los resultados corregidos también debían poder salir de la aplicación sin errores. Para ello se solucionaron los problemas al exportar tablas en CSV y JSON y se agregó la exportación en formato vectorial EMF, mediante un escritor propio que genera los ejes, rótulos y líneas como elementos vectoriales e incorpora los mapas de calor como imágenes. Por defecto, los archivos exportados se guardan en la carpeta *archivos/* del proyecto abierto.

Finalmente, la forma de comunicar estos resultados también influyó en la estabilidad. Los avisos sobre los resultados de los cálculos dejaron de abrir ventanas modales, porque este tipo de ventanas genera un bucle de eventos adicional que podía provocar cierres inesperados cuando se seguían ejecutando los temporizadores de VTK. En su lugar, se utilizan avisos que no bloquean la interacción del usuario con la aplicación.

Nuevas funcionalidades Las mejoras anteriores hacen que una sesión de trabajo sea estable y confiable, pero en Gamma Lab 1.0 esa sesión se perdía al cerrar la aplicación, junto con los *trials*, el preprocesamiento, las mediciones y la configuración de los análisis. Para solucionarlo, Gamma Lab 2.0 incorpora *ProjectService* y un nuevo formato de proyecto llamado *.glab*. Cada proyecto se organiza en una carpeta con tres elementos: un manifiesto en formato JSON con extensión *.glab*, una copia del archivo de señal original y una subcarpeta *archivos/*, donde se almacenan las exportaciones. Al incluir la copia de la señal, el proyecto puede trasladarse a otro equipo sin depender de la ubicación original del archivo.

El manifiesto aplica el mismo principio de las recetas: en lugar de guardar las matrices de datos, almacena la información necesaria para reconstruirlas al abrir el proyecto. Así, los *trials* se guardan mediante sus parámetros de recorte, los índices de los elementos descartados y las recetas de preprocesamiento aplicadas; los análisis, mediante los parámetros de cada

extensión que tenga habilitada la opción *saves_analysis: true*; y la frecuencia de muestreo, junto con su origen, lo que permite identificar si fue modificada manualmente. Las mediciones son la excepción y se almacenan completas, porque contienen pocos datos y dependen de puntos que el usuario marca manualmente. Con esta estructura, los archivos de proyecto son más livianos, fáciles de consultar y cuentan con una versión de esquema (*schema_version*). Además, al reconstruir los datos con el mismo código de procesamiento, se busca obtener los mismos resultados que se tenían antes de cerrar el proyecto.

ProjectService también protege la integridad del proyecto. El estado *dirty* permite identificar si existen cambios sin guardar, y el servicio impide guardar proyectos directamente en carpetas del sistema, como Escritorio, Documentos o Descargas. Asimismo, evita copiar nuevamente la señal cuando ya se encuentra dentro del proyecto y, cuando sí debe copiarla, lo hace por bloques en un archivo temporal que solo se renombra al finalizar correctamente, de manera que una copia cancelada no deja un archivo incompleto. Finalmente, el sistema operativo puede abrir directamente los archivos *.glab* utilizando Gamma Lab.

Flujo general de la aplicación Los componentes descritos actúan de forma conjunta a lo largo de una sesión de trabajo. Cuando se inicia Gamma Lab, *main.py* crea la aplicación Qt, muestra la pantalla de bienvenida, crea el *Kernel* y registra los servicios principales: *DataStore*, *FileIO* y *TaskService*. Este registro se realiza antes de cargar las extensiones, ya que estas pueden necesitar dichos servicios desde su método *initialize*. Después, *PluginManager* recorre la carpeta *plugins/*, crea las instancias de cada extensión y las registra en el *Kernel*. Por último se construye *MainWindow*, que organiza las extensiones en la barra de categorías de la interfaz y se registra como un servicio adicional. Si Gamma Lab se inició directamente desde un archivo *.glab*, el sistema carga también el proyecto correspondiente.

A partir de ese momento, el usuario puede abrir archivos de señal en formatos *.abf*, *.edf* y *.mat*, o un proyecto completo en formato *.glab*. En este segundo caso, *MainWindow* verifica primero si existen cambios sin guardar o cálculos en ejecución, y luego solicita a *TaskService* el proceso de preparación mediante *preparar_apertura()*. Un hilo de trabajo lee el manifiesto y el archivo de señal a través de *FileIOService*, sin modificar todavía el *DataStore* ni la interfaz gráfica, conforme a la regla de escritura descrita anteriormente. Solo cuando la lectura finaliza correctamente se libera el proyecto anterior y se ejecuta, desde el hilo de la interfaz, el método *open()*. Este registra la nueva señal como activa, aplica la frecuencia de muestreo corregida, reconstruye los *trials* a partir de sus parámetros de recorte y recupera los descartes y luego las recetas de preprocesamiento sobre el conjunto derivado; este orden es importante, porque cada receta conserva los descartes vigentes cuando fue aplicada. Finalmente, se restauran las mediciones y se envían los parámetros guardados a las extensiones correspondientes. Si la

lectura falla o el usuario la cancela, el proyecto que estaba abierto se mantiene sin cambios.

Con la señal cargada, el usuario puede generar *trials*, realizar tareas de preprocesamiento, como filtrado, eliminación de artefactos y ajuste de la frecuencia de muestreo, y ejecutar los análisis disponibles. Antes de iniciar cada análisis, el sistema valida los parámetros ingresados, cancela cualquier cálculo anterior de la misma extensión y envía la nueva tarea a *TaskService*. Cuando el cálculo termina correctamente, el resultado llega al hilo de la interfaz, se muestra mediante VTK y el proyecto se marca como modificado. Los resultados numéricos permanecen en la extensión que realizó el análisis, mientras que el *DataStore* conserva únicamente la información compartida entre los componentes: las señales, la señal activa, las mediciones y el *trial* en curso.

Cuando el usuario decide conservar ese trabajo, el método *iniciar_guardado()* verifica que exista una señal activa y determina la carpeta del proyecto, que puede ser una ubicación utilizada anteriormente o una nueva carpeta seleccionada y validada por el usuario. Si es necesario copiar el archivo de señal, la operación se realiza mediante *TaskService*, lo que permite consultar el progreso y cancelarla. Cuando la copia termina correctamente, *completar_guardado()* genera el manifiesto a partir de los *trials*, las mediciones y los parámetros de cada extensión, y el estado cambia a *dirty* = *False*, indicando que todos los cambios han sido guardados. Si la copia falla o se cancela, el sistema revierte el estado de la operación y evita generar un archivo *.glab* incompleto.

La sesión termina al cerrar el proyecto o la ventana principal, y en ambos casos se utiliza el mismo mecanismo de confirmación para evitar la pérdida de información. Si hay un cálculo en ejecución, se pregunta al usuario si desea continuar, lo que implica cancelar las tareas pendientes; si existen cambios sin guardar, se muestran las opciones *Save*, *Don't Save* y *Cancel*. Una vez autorizado el cierre, *ProjectService.close()* limpia el estado del proyecto y *liberar_senal_anterior()* cancela las tareas, cierra los recursos de VTK, destruye los *widgets*, desvincula las extensiones de la señal y libera los datos almacenados en el *DataStore*. La aplicación regresa entonces a la pantalla de inicio, con la memoria disponible para cargar una nueva señal, cerrando el problema de acumulación que presentaba Gamma Lab 1.0. Cuando se cierra completamente la aplicación, también se ejecuta el método *stop()* de cada extensión.

La siguiente tabla resume los principales avances de Gamma Lab 1.0 a Gamma Lab 2.0.

Aspecto	Gamma Lab 1.0	Gamma Lab 2.0
Persistencia	No permitía guardar proyectos	Incorpora proyectos *.glab* que pueden reconstruirse y trasladarse entre equipos
Memoria	Las señales anteriores permanecían almacenadas	Incorpora *release_signals()* para liberar datos al cambiar de señal o cerrar un proyecto
Ejecución	Los análisis se ejecutaban principalmente en el hilo de la interfaz	Incorpora *TaskService* para gestionar tareas, consultar su progreso y cancelarlas
Concurrencia de datos	No existía un control centralizado	El *DataStore* utiliza un único escritor y un mecanismo de bloqueo
Filtros	Chebyshev y elíptico fallaban porque no estaban implementados	Ofrece Butterworth en segundo plano; los filtros internos usan SOS con validaciones y memoización
Wavelet	Presentaba problemas en la normalización	Incorpora una línea base por frecuencia y utiliza la tasa de muestreo efectiva
Artefactos	Los cambios se aplicaban directamente sobre los datos originales	Utiliza conjuntos derivados y recetas para reproducir los cambios
Exportación	Presentaba fallas en CSV y JSON y no ofrecía exportación EMF	Corrige la exportación CSV y JSON e incorpora el formato vectorial EMF
Análisis	Incluía análisis en tiempo, frecuencia y tiempo-frecuencia	Incorpora PAC (en uno y dos canales) y PPC

Tabla 5: Principales mejoras de Gamma Lab 1.0 a Gamma Lab 2.0

5.3.2 Análisis de señales

Esta sección presenta el diseño de la ampliación del alcance analítico de Gamma Lab 2.0, es decir, de los módulos que permiten estudiar el acoplamiento entre ritmos de distinta frecuencia y la sincronización de fase entre canales. Los demás frentes de la solución solo se mencionan cuando estos módulos dependen de ellos.

Técnicas incorporadas

La ampliación agrega tres módulos al Subsistema de Análisis: PAC (*phase-amplitude coupling*), que reúne cuatro modos de cálculo; el índice de modulación; y PPC (*phase-phase coupling*), que evalúa el acoplamiento de fase entre dos canales. Todos parten de análisis

disponibles en el sistema de referencia BOARD-FTD-PACC [7] y cubren los requerimientos R13 a R23, R60 a R66, R74 y R75. La Tabla 6 resume la pregunta que responde cada técnica.

Tabla 6: Técnicas de la ampliación del alcance analítico

Técnica	**Pregunta que responde**	***Trials* que usa**	**Salida principal**
PAC	¿En qué punto del ciclo lento aumenta la energía rápida de un *trial*?	Uno, elegido por el usuario	Mapa fase-frecuencia, histograma de fase, vector medio y cuatro señales
PAC *Average*	¿Se mantiene ese patrón sobre el conjunto de *trials*?	Todos los válidos	Mapa, histograma y vector medio
PAC Psel	¿Cómo se comporta la actividad rápida alrededor de los ciclos completos del ritmo lento?	Todos, concatenados	Trece paneles con ciclos superpuestos, promedios, mapas y vectores
PAC Psel inter-canal	¿El ritmo lento de un canal organiza la actividad rápida de otro?	Todos, de dos canales	Los mismos paneles de PAC Psel
Índice de modulación	¿Qué pares de bandas de fase y de amplitud están acoplados?	Uno, elegido por el usuario	Comodulograma
PPC	¿La diferencia de fase entre dos canales se mantiene estable? ¿En qué frecuencia y en qué momento?	Todos, emparejados entre canales	Mapa frecuencia-tiempo

Criterios de diseño

Cinco criterios orientaron las decisiones de este diseño:

- **Un módulo por técnica.** PAC, PAC *Average* y PAC Psel comparten parámetros, por lo que son modos de un mismo módulo. El índice de modulación usa otro algoritmo y PPC mide otra relación, de modo que cada uno es un módulo independiente.
- **Cálculo separado de la presentación.** Cada módulo distingue dos responsabilidades: realizar el análisis y atender al usuario. El cálculo recibe datos y parámetros y entrega un resultado, sin conocer la interfaz ni las gráficas. Así puede ejecutarse en segundo plano y verificarse de forma independiente.
- **Lo compartido se ubica en el núcleo.** Un componente de cálculo que necesitan dos módulos pasa al núcleo, y el que necesita uno solo permanece en ese módulo.
- **Paridad con la referencia y diferencias justificadas.** El diseño busca reproducir el cálculo de BOARD-FTD-PACC, salvo cuando hacerlo conduciría a un resultado engañoso. Esas diferencias son deliberadas y se recogen en la Sección 5.3.2.

- **Ningún resultado dudoso sin advertencia.** Lo que no se puede medir queda sin valor en lugar de mostrar un número, y un parámetro imposible se rechaza con un mensaje en lugar de ajustarse en silencio.

Componentes y responsabilidades

La ampliación se organiza en tres niveles de responsabilidad, coherentes con la arquitectura de *plugins* de la plataforma:

- **Módulos de análisis.** Cada técnica es un *plugin* que se integra sin modificar el núcleo (R77). Su parte de coordinación recibe los parámetros, valida las entradas, solicita el cálculo y presenta el resultado. Su parte de cálculo contiene el algoritmo propio de la técnica.
- **Servicios del núcleo.** Los módulos no resuelven por sí mismos lo transversal: se apoyan en los servicios que ejecutan tareas en segundo plano, dan acceso a los *trials*, exportan resultados y guardan el proyecto.
- **Cálculo compartido.** Las operaciones que usan varios módulos forman un componente común, de modo que cada una tiene una sola definición (Tabla 7).

Tabla 7: Componentes de cálculo compartidos

Componente	**Responsabilidad**	**Lo usan**
Filtrado	Aislar una banda de frecuencia sin introducir desfase	PAC, índice de modulación
Transformada de Gabor	Obtener la representación tiempo-frecuencia de una señal	PAC Psel, PPC
Tasa efectiva	Determinar la frecuencia de muestreo real después del submuestreo	PAC, índice de modulación, PPC
Reparto de *trials*	Distribuir los *trials* para procesarlos en paralelo, según la memoria disponible	PAC *Average*, PPC

La *wavelet* de Morse [10] permanece dentro de PAC porque ningún otro módulo la necesita.

Datos de entrada y resultados

Los módulos reciben los *trials* activos del canal: una matriz de muestras por *trials* que excluye los *trials* descartados y que, si el canal fue preprocesado, corresponde al conjunto derivado (R79). PAC y el índice de modulación trabajan sobre el canal activo, mientras que el modo inter-canal y PPC solicitan cada canal por su nombre (R68).

Un análisis no modifica la señal ni los *trials*. Su resultado reúne las matrices, los ejes y los datos de contexto, como la tasa efectiva y el número de *trials* o de ciclos. El módulo lo conserva junto con los parámetros de la ejecución, lo que permite exportarlo sin recalcular.

Diseño de cada técnica

PAC y PAC *Average*. PAC estima en qué punto del ciclo de un ritmo lento aumenta la energía de las frecuencias rápidas. Sobre un *trial* ya submuestreado, el cálculo sigue estos pasos:

1. Se filtra la señal en la banda de fase con un filtro Chebyshev II bidireccional y se obtiene la fase instantánea $\phi(t)$ con la transformada de Hilbert.

2. Se calcula la energía rápida con una *wavelet* de Morse generalizada [10], que produce una matriz $W(f, t)$ con cuatro filas por cada Hz del rango de amplitud.

3. Se divide el ciclo en $K = 100$ intervalos de fase y se promedia la magnitud de W en los instantes T_k cuya fase cae en cada intervalo:

$$\bar{A}(f, k) = \frac{1}{|T_k|} \sum_{t \in T_k} |W(f, t)|$$

4. Se normaliza cada frecuencia con un puntaje z a lo largo de los intervalos. Así, las frecuencias de poca energía se leen igual que las fuertes, porque lo que interesa es si varían con la fase.

5. Se resume el acoplamiento en el vector medio, a partir del histograma de fase h_k:

$$v = \frac{1}{K} \sum_{k=1}^{K} h_k \, e^{i\theta_k}$$

   donde θ_k es la fase del intervalo k. Su magnitud indica la intensidad del acoplamiento y su ángulo, la fase en que ocurre.

PAC *Average* usa el mismo algoritmo, pero acumula las sumas y las cuentas de cada intervalo sobre todos los *trials* y divide una sola vez al final. Como cada *trial* es independiente, los *trials* pueden procesarse en paralelo, dentro de los límites de la memoria disponible.

El mapa, el histograma y el vector se presentan juntos. PAC agrega la señal original, la banda de fase, la fase instantánea y la banda de amplitud sobre un eje de tiempo común. PAC *Average* no muestra esas señales, porque presentar un *trial* bajo un título de promedio invitaría a leerlo como promediado.

PAC Psel. PAC Psel analiza ciclos completos del ritmo lento. Concatena los *trials* en una señal continua, la filtra en la banda de fase y localiza sus picos, valles y cruces por cero. Un valle se acepta cuando a su alrededor aparece la secuencia pico, cruce, valle, cruce y pico, sus picos superan la media de los picos, el valle queda por debajo de la media de los valles y cada semiperiodo tiene una duración compatible con la banda elegida.

Alrededor de cada valle aceptado se recorta una ventana y las ventanas se superponen, lo que permite comprobar visualmente si la actividad rápida se alinea con el ciclo (R75). El promedio de esas ventanas delimita el tramo entre sus dos primeros picos, y sobre ese tramo se agrupa la energía por fase. Este modo usa la *wavelet* de Gabor, con dos filas por Hz, y un panel de actividad rápida fijo en la banda de 25 a 129 Hz, como la referencia. Los trece paneles se presentan en una sola columna, en lugar de las ventanas separadas del sistema original.

PAC Psel inter-canal. Este modo aplica el diseño de PAC Psel a dos canales: los ciclos y la fase salen del canal activo, y la amplitud sale del canal que elige el usuario (R60 a R62). La segunda fuente de datos es la única diferencia entre ambos modos. Como en la referencia, los umbrales de selección de ciclos se relajan en una desviación estándar. El panel de actividad rápida se toma del canal de amplitud, decisión propia de este diseño, para que los paneles se lean juntos.

Índice de modulación. Este módulo barre una rejilla de bandas para mostrar qué pares están acoplados. Para cada frecuencia de fase filtra una banda de 1 Hz y para cada frecuencia de amplitud, una de 10 Hz; con la transformada de Hilbert obtiene la fase ϕ y la envolvente a. Después calcula, para cada par, el índice de Canolty [8] y su variante normalizada [9]:

$$MI = \left| \frac{1}{N} \sum_{n=1}^{N} a(n)\, e^{i\phi(n)} \right| \qquad MI_{norm} = \frac{\left| \sum_{n=1}^{N} a(n)\, e^{i\phi(n)} \right|}{\sum_{n=1}^{N} a(n)}$$

El cruce entre todas las bandas se plantea como una sola operación matricial y no par por par, porque con los valores por defecto la rejilla reúne cerca de cien mil combinaciones. Los pasos de la rejilla solo mueven el centro de cada banda, no su ancho. La gráfica muestra MI_{norm} y la exportación incluye las dos medidas.

Acoplamiento fase-fase entre canales (PPC). PPC evalúa si la diferencia de fase entre dos canales se mantiene estable a una misma frecuencia. Para cada *trial*, ambos canales se transforman con una *wavelet* de Gabor, de tres ciclos por omisión, sobre un eje logarítmico de frecuencias, y se conserva solo la fase. La referencia cuantifica esa estabilidad con el PLV. Este diseño usa en cambio la consistencia de fase por pares C [11], que corrige el sesgo debido

al número de muestras N:

$$C = \frac{N \cdot PLV^2 - 1}{N - 1}$$

Esa medida también se abrevia PPC en la literatura (*pairwise phase consistency*), pero en este documento la sigla designa el análisis de acoplamiento fase-fase.

El valor de N cuenta solo las muestras independientes, que son las separadas al menos dos desviaciones estándar de la *wavelet*. Cada frecuencia tiene su propia ventana, dimensionada para reunir 20 muestras independientes entre todos los *trials*, con un mínimo configurable de 50 ms por omisión y un máximo del *trial* completo. Junto a los extremos del *trial* la ventana se recorta y N disminuye en la misma proporción. Si una celda no reúne dos muestras independientes, queda sin valor.

Cada *trial* se procesa de forma independiente y los aportes se suman en un orden fijo, por lo que el resultado no depende de cuál termine primero. PPC exige que los dos canales provengan del mismo corte: si uno no tiene *trials*, el módulo los genera con los parámetros del otro.

Flujo de ejecución

Todos los módulos siguen el flujo de la Figura 7. El módulo lee y valida los datos y solicita el cálculo al servicio de tareas del núcleo, que lo ejecuta en segundo plano, una tarea a la vez. Mientras tanto, el cálculo informa su avance y la aplicación sigue respondiendo.

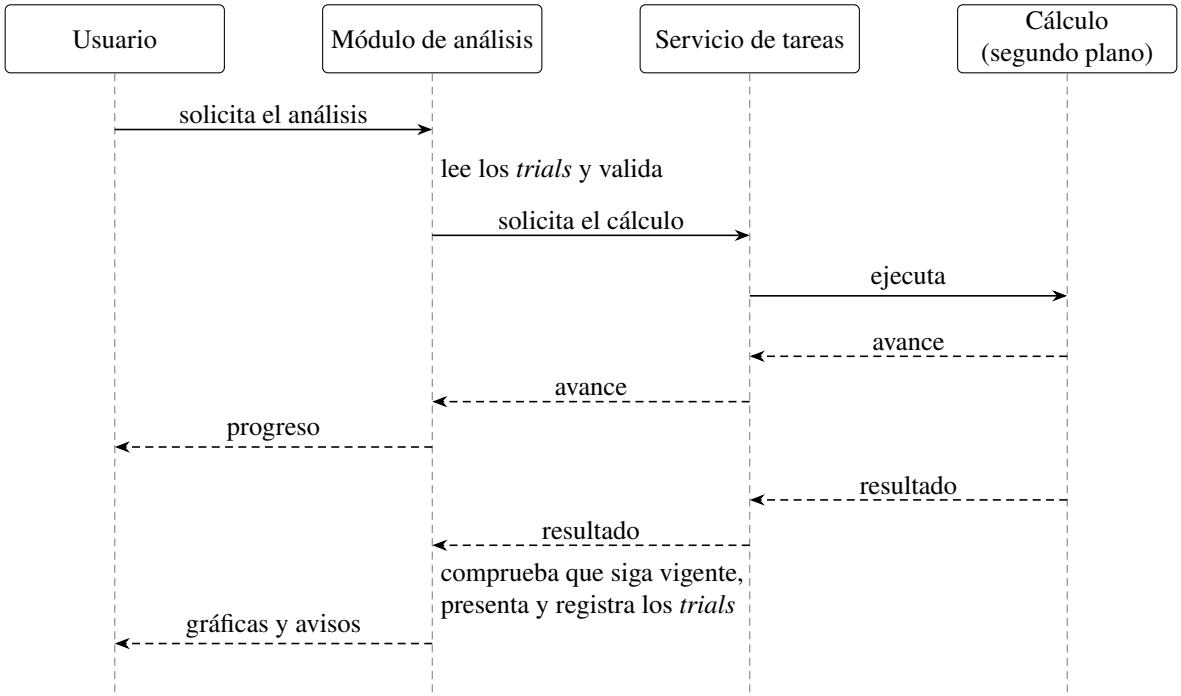


Figura 7: Flujo de ejecución de un análisis

Durante la ejecución, la tarea comunica cuatro eventos al módulo: avance, finalización, fallo y cancelación. Antes de atender cada uno, el módulo comprueba que la tarea siga siendo la vigente, porque al cambiar un parámetro se solicita otra y la anterior puede seguir activa un instante; sin esa comprobación, su resultado sobrescribiría al nuevo.

La cancelación es cooperativa: el cálculo la consulta entre etapas y entre *trials*. Puede pedirla el usuario desde la barra de progreso, y también ocurre al salir del módulo o al lanzar otro cálculo del mismo módulo. Si la tarea no responde en dos segundos, el servicio deja de esperarla, descarta su resultado y continúa con la siguiente.

Diferencias deliberadas con la referencia

La Tabla 8 recoge los puntos en que el diseño se aparta de BOARD-FTD-PACC. En todos ellos, replicar la referencia habría producido un resultado engañoso o la interrupción del análisis.

Tabla 8: Diferencias deliberadas con BOARD-FTD-PACC

Aspecto	**Referencia**	**Gamma Lab 2.0**	**Motivo**
Medida de PPC	PLV	Consistencia de fase por pares	El PLV depende del número de muestras: con fases al azar da 0,63 con 2 muestras y 0,02 con 2000
Muestras contadas	Todas las de la ventana	Solo las independientes	Contar muestras correlacionadas acerca el valor a 1: una senoidal contra ruido blanco dio 0,99993
Ventana de PPC	Una para toda la banda	Una por frecuencia	Una ventana fija no alcanza en frecuencias bajas y sobra en las altas
Celdas no medibles en PPC	Valen 1	Quedan sin valor	Un 1 se lee como acoplamiento perfecto
Frecuencia sobre el Nyquist	Se recorta en silencio	Se rechaza con un mensaje	Quien pide un rango debe saber si el análisis lo cubre
Bandas del comodulograma	Tope fijo de 178 bandas	Barrido completo, salvo las bandas junto al Nyquist	Con los valores por defecto, el tope dejaba vacío el 81,9 % de las filas
Ciclos junto al borde en PAC Psel	Detienen el análisis	Se omiten y se informa cuántos	Un ciclo del borde no debe detener todo el análisis
Señales mostradas en PAC	Con un desfase acumulado	Sin desfase	El desfase contradecía al mapa que aparece junto a ellas

Persistencia y exportación

El proyecto guarda la receta de cada análisis y no sus resultados: los parámetros y los *trials* usados (R23). Al abrirlo, el sistema vuelve a cortar los *trials* y recalcula, de modo que el proyecto se mantiene liviano y el resultado siempre proviene de la señal. Cada módulo declara si participa en este mecanismo; PAC y PPC lo hacen, y ese recálculo no marca el proyecto como modificado.

Para exportar, cada módulo define las tablas de su resultado y el servicio de exportación del núcleo las escribe (R21, R65). PAC entrega parámetros, resumen, histograma y mapa; el índice de modulación, parámetros, resumen y las dos matrices; y PPC, parámetros, resumen, ventana por frecuencia, mapa y avisos. Los formatos disponibles son CSV, Excel y JSON. Las gráficas se exportan como imagen desde el mismo menú contextual de los demás análisis (R22, R66).

Control de entradas y avisos

La Tabla 9 resume cómo responde cada módulo ante datos o parámetros que comprometen el resultado. Los errores que detienen el cálculo se muestran en un diálogo. Las advertencias sobre un resultado ya calculado no interrumpen al usuario: PPC las muestra bajo la gráfica y el índice de modulación las deja en el registro de la aplicación.

Límites del diseño

El diseño tiene los siguientes límites conocidos:

- **Sin prueba de significancia.** Los módulos calculan la magnitud del acoplamiento, pero no la comparan contra datos sustitutos. Esa valoración queda a cargo del investigador, con los valores exportados, y debe hacerse con cuidado porque puede aparecer acoplamiento aparente sin interacción real [12].
- **Submuestreo sin filtro *antialias*.** Se conserva para coincidir con la referencia, por lo que el contenido sobre el nuevo Nyquist se pliega sobre frecuencias bajas.
- **Cancelación no inmediata.** Una etapa en curso no se puede interrumpir. Pasados dos segundos el resultado se descarta, pero el cálculo puede seguir activo hasta terminar, lo que deja parcial a R46.
- **PAC entre canales solo en modo Psel.** Los modos PAC y PAC *Average* trabajan sobre un canal.
- **Índice de modulación sin persistencia.** Este módulo todavía no participa del mecanismo de guardado, por lo que su análisis no se conserva con el proyecto.

Tabla 9: Respuesta ante entradas que comprometen el resultado

Condición	**Respuesta**
Frecuencia alta sobre el Nyquist efectivo (PAC, PPC)	Se rechaza y el mensaje indica el límite
Bandas del comodulograma junto al Nyquist	Esas bandas quedan sin valor, se calcula el resto y queda constancia en el registro
Densidad de muestreo que no divide la tasa del archivo	Todos los ejes usan la tasa efectiva, obtenida con un factor entero; PPC lo advierte bajo la gráfica
Menos de tres ciclos utilizables (PAC Psel)	Se rechaza y se sugiere ampliar la banda o usar más *trials*
El mismo canal dos veces (PAC Psel inter-canal, PPC)	Se rechaza antes de iniciar el cálculo
Trials de dos canales no alineados	No se calcula: PPC compara forma, inicio de cada *trial*, tasa y ventana, y PAC Psel inter-canal compara la forma de las matrices
Trial sin variación (PPC)	Se rechaza, porque normalizar ruido numérico daría un acoplamiento perfecto falso
Frecuencias sin dos muestras independientes (PPC)	Quedan sin valor y se advierte qué rango no se midió
Frecuencias con menos muestras de las pedidas (PPC)	Se advierte bajo la gráfica y en la exportación
Menos de seis ciclos de la banda lenta (índice de modulación)	Queda constancia en el registro; el cálculo no cambia

- **Aproximación de las muestras independientes en PPC.** Con ruido independiente, la media medida fue de $-0{,}009$ en lugar de 0. El sesgo va hacia abajo, de modo que no crea acoplamiento.
- **Tramos planos en PPC.** Se detecta un *trial* constante, pero no un tramo plano dentro de un *trial* por lo demás activo.
- **Dependencia entre módulos.** PPC reutiliza una operación de presentación de PAC, la única excepción al criterio de ubicar en el núcleo lo que comparten dos módulos.

5.3.3 Interfaz de usuario (UI/UX)

File

Se añadió el apartado *File* como primera opción de la barra de menú superior, antes de *Home*, *Preprocessing* y *Analysis*. Su propósito es reunir en un solo lugar las acciones que afectan a la sesión de trabajo completa y no a un análisis concreto. Su posición responde a la convención de las aplicaciones de escritorio, en las que el usuario espera encontrar en

este menú las operaciones para abrir, guardar y cerrar su trabajo. A diferencia de las demás categorías, que presentan sus herramientas en la barra de *plugins*, *File* se despliega como un menú breve de seis opciones: *Open signal*, *Open project*, *Save*, *Save as*, *Close project* y *Exit*. El orden acompaña el ciclo natural de una sesión: primero se abre una señal o un proyecto, después se guarda el trabajo y, por último, se cierra el proyecto o se sale de la aplicación. La incorporación de este apartado está ligada al concepto de proyecto. Un proyecto se guarda en una carpeta propia e incluye un archivo con extensión .glab junto con una copia de la señal.

Home

En la versión 1.0 de GammaLab, al abrir la aplicación, el usuario no recibía ninguna explicación del funcionamiento general. Tenía que descubrir por su cuenta el orden de uso de las secciones (*Home*, *Preprocessing* y *Analysis*) y la existencia de funciones accesibles solo desde el clic derecho sobre una gráfica. Esto es parte de lo que dificultaba construir un modelo mental claro de la herramienta.

En la actualidad GammaLab se diseñó para que "*Home*" fuera la primera sección que viera el usuario y la que se muestra por defecto. Concentra las acciones básicas para trabajar de esta manera el usuario conoce que funciones existen directamente con una señal ya cargada, como lo son medirla y ajustar la vista de la gráfica. Sus botones se organizan en dos grupos: *Measurements* (pendiente y amplitud, con menús para eliminar y para mostrar u ocultar mediciones) y *Zoom* (ventana emergente que actúa sobre la última gráfica visitada). El botón de la herramienta activa queda resaltado con un contorno, para que el usuario sepa en todo momento qué está usando.

Para orientar al usuario nuevo se añadieron tres elementos. El primero es un panel de bienvenida en el área central, con el logotipo y la instrucción "*To get started, open a signal*". Se muestra solo en el *Home* y mientras no haya una herramienta activa. El segundo es una guía que aparece la primera vez que se ejecuta la aplicación, con el recorrido en cinco pasos (*Home*, *Preprocessing*, *Analysis*, las barras *Explorer* y *Results*, y el clic derecho sobre una gráfica). El tercero son mensajes explicativos por cada funcionalidad para que el usuario sepa qué hace cada herramienta antes de usarla: al pasar el cursor sobre un botón de la cinta aparece su nombre y una descripción breve de su función, y lo mismo ocurre con cada control de los paneles de parámetros.

Preprocessing

La sección *Preprocessing* reúne las herramientas que preparan la señal antes del análisis: segmentarla en *trials*, filtrarla y retirar artefactos. Lo que se hace aquí determina los datos sobre los que trabajan las herramientas de *Analysis*.

Los botones se agrupan en cuadros etiquetados, igual que en el *Home*, y siguen el orden

del flujo de trabajo, de izquierda a derecha. El grupo *Trials* contiene la herramienta que segmenta la señal. El grupo *Prepare* contiene *Filter* y *Remove Artifact*, que modifican los *trials* ya generados. Cada botón muestra al pasar el cursor su nombre y una descripción breve de su función.

Antes, al generar los *trials*, el resto de la aplicación no se enteraba, y el usuario no tenía forma de saber qué *trial* estaba revisando. Ahora, al generarlos, la herramienta avisa a la ventana principal, que muestra un indicador con el *trial* visible, por ejemplo, su posición entre los válidos. El indicador dice ”Sin *trials* generados” si aún no hay, y señala cuando el *trial* visible está descartado. Se actualiza al cambiar de herramienta. La barra de estado confirma cada acción, como cuántos *trials* se generaron y con qué umbral, o qué *trial* se está revisando. *Previous* y *Next* recorren los *trials* de forma circular, de modo que el usuario nunca llega a un extremo sin salida.

Barra Parameters

La barra *Parameters* es el panel de la derecha, que cada herramienta de *Preprocessing* y *Analysis* usa para configurar y lanzar su cálculo. Se diseñó con la misma estructura en todas las herramientas, para que lo aprendido en una sirva en las demás.

Estructura. El panel tiene dos zonas. La cabecera es fija, nunca se desplaza, y contiene el título ”*Parameters*”, el botón de limpiar parámetros y el botón que ejecuta la herramienta (por ejemplo, *Generate*). Debajo, una línea divisoria la separa del cuerpo, que reúne los controles en secciones y se desplaza solo si el panel queda estrecho, por ejemplo cuando se abre el panel de resultados. Así, el botón de ejecutar y el de limpiar siempre están a la vista, sin importar cuántos parámetros tenga la herramienta.

GammaLab 2.0 incorporó tres mejoras a esta barra:

1. **Descripción emergente en cada control.** Al dejar el cursor sobre un campo, lista o botón, aparece una explicación breve. Por ejemplo, ”Frecuencia máxima del rango de análisis, en Hz” o ”Orden del filtro. Un orden mayor corta con más pendiente”. Un servicio recorre los controles de cada módulo al abrirlo y completa los que no traen descripción propia. Usa un diccionario con las descripciones de los parámetros habituales y, si el control no está ahí, el rótulo que lo acompaña. Nunca reemplaza una descripción que el módulo ya declaró. De este modo ningún control queda sin explicación, incluso en los módulos que se agreguen después.

2. **Selector de *trials*.** Las herramientas de análisis añaden al panel una lista con una casilla por *trial* válido, con los botones *All* y *None*. Los *trials* descartados no aparecen. Algunas

herramientas solo calculan sobre un *trial*; en ellas la lista permite marcar uno a la vez, y al marcar uno se desmarcan los demás. Así el usuario controla sobre qué datos se calcula sin salir del panel.

3. **Limpiar restablece todo.** El botón de limpiar devuelve al valor inicial todos los controles del panel, incluidos los selectores de canal, de canal de estímulo y de modo de fin de *trial*. *Open Signal* añadió además un botón *Clear* que descarta el valor escrito y muestra de nuevo la frecuencia de muestreo vigente.

Barra de búsqueda

Para la primera versión de la aplicación, el usuario tenía que saber en qué sección estaba (*Home*, *Preprocessing* o *Analysis*), cambiar a esa sección y buscar el botón en la cinta. Esto exige haber construido de antemano un modelo mental de dónde está cada función, algo difícil para quien usa la aplicación por primera vez. Ahora, existe una barra de búsqueda visible desde cualquier sección. Lleva un ícono de lupa y el texto guía "*search function by name*", que indica qué se puede escribir. Funciona así:

1. Al teclear, aparece una lista con los nombres de las herramientas que contienen el texto, sin distinguir mayúsculas de minúsculas. Basta escribir un fragmento, por ejemplo "wav" para encontrar las herramientas de *wavelet*, sin recordar el nombre completo.

2. Al elegir una sugerencia, la herramienta se abre y el campo se limpia, listo para la siguiente búsqueda. Si el usuario escribe el nombre exacto y pulsa *Enter*, también se abre.

3. Las sugerencias se calculan a partir de las herramientas registradas en el núcleo de la aplicación. Cuando se agrega una herramienta nueva, aparece en la búsqueda sin modificar la interfaz, en coherencia con la arquitectura de *plugins*.

Analysis

La sección *Analysis* reúne las técnicas que se calculan sobre los *trials* ya preparados. Es la más extensa de la aplicación, por lo que su diseño buscó que el usuario encontrara cada técnica con rapidez y supiera siempre sobre qué datos se calculó cada resultado.

En la versión 1.0 de GammaLab, los botones de las técnicas no tenían un orden definido por dominio. Ahora se agrupan en cuatro cuadros etiquetados, de izquierda a derecha, por tipo de análisis: *Time* (*Average* y *Event Related Potential*), *Frequency* (FFT, FFT *Average*, PSD, PSD *Average* y *Relative* PSD), *Time-Frequency* (PAC, *Modulation Index*, *Wavelet*, *Wavelet Average* y PPC). El orden de los grupos y de los botones dentro de cada grupo lo declara

cada herramienta en su descriptor, de modo que una técnica nueva se coloca sin modificar la interfaz. Cada botón muestra al pasar el cursor su nombre y una descripción de lo que calcula y de las opciones que ofrece.

Clasificación de avisos

En la clasificación anterior, todos los avisos de la aplicación eran el mismo cuadro de diálogo modal del sistema (*QMessageBox*). Una confirmación como "*Image exported successfully*" interrumpía al usuario igual que un error: había que cerrar la ventana antes de seguir trabajando. Lo único que distinguía un aviso de otro era el ícono estándar del sistema. En la práctica, el usuario aprendía a cerrar todos los mensajes sin leerlos, y los importantes se perdían entre los informativos. Ahora los avisos se clasifican en tres niveles de severidad, y cada nivel se presenta de forma distinta según cuánto debe interrumpir al usuario:

Nivel	**Color**	**Presentación**	**Duración**
Información	Azul	Tarjeta no modal en la esquina inferior izquierda del área de trabajo	Se cierra sola a los 4,5 s
Advertencia	Ámbar	Tarjeta no modal en el mismo lugar	Permanece hasta que el usuario la cierra
Error	Rojo	Cuadro de diálogo modal	Exige que el usuario lo atienda

Tabla 10: Niveles de severidad de los avisos en GammaLab 2.0.

La lógica detrás de esta clasificación es que el aviso debe interrumpir tanto como lo merezca lo ocurrido. Una confirmación de éxito no requiere acción, así que desaparece sola y no estorba. Una advertencia, por ejemplo "Marca al menos un *trial* para calcular", sí requiere que el usuario la note, pero no detiene su trabajo, así que se queda visible sin bloquear la ventana. Un error detiene lo que el usuario estaba haciendo, así que es el único que bloquea hasta que lo atiende.

Organización. Las tarjetas aparecen abajo a la izquierda porque en todas las herramientas el panel de parámetros está a la derecha. Así los avisos caen sobre la gráfica y no tapan los controles. Se apilan con el más reciente abajo, y se muestran como máximo cuatro a la vez: cuando hay más, se descartan primero los informativos más antiguos y las advertencias se conservan.

Barra lateral: Explorer y Results

Para fortalecer la navegación se diseñó una barra lateral ubicada en el costado izquierdo de la ventana principal. Esta barra concentra los accesos a los recursos que pertenecen a todo

el proyecto y no a un análisis en particular: los archivos del proyecto (*Explorer*) y las tablas de mediciones (*Results*). Antes de este diseño, el usuario debía buscar esta información en menús o ventanas separadas. Con la barra lateral, ambos recursos quedan a un clic de distancia desde cualquier punto de la aplicación.

La barra se organizó en torno a una columna estrecha de iconos que permanece siempre visible. En la parte superior se ubican los accesos a *Explorer* y *Results*, que son las herramientas de trabajo, y en la parte inferior se separa el acceso a *Help*, que cumple una función de soporte. La columna sirve como punto de referencia fijo y, gracias a su reducido ancho, cede casi todo el espacio a las señales y gráficas. Para no ocupar espacio con etiquetas permanentes, cada botón presenta un icono y muestra su nombre al pasar el cursor, momento en el que también se resalta con un azul claro que indica que es interactivo.

Explorer. El *Explorer* se concibió como un panel de consulta y organización de archivos. Se abre y se cierra con el mismo botón, y su ancho se ajusta arrastrando el borde. Al reabrirlo conserva el ancho elegido, mientras que la primera vez se abre con un tamaño que permite leer los nombres de los archivos. Cuando cambia de tamaño, el área de trabajo y la barra de herramientas se reacomodan juntas, de manera que ningún elemento queda oculto. El panel muestra la carpeta real del proyecto activo, se actualiza al abrir, guardar, renombrar o cerrar un proyecto y excluye las carpetas ajenas. Además, presenta únicamente el nombre de cada elemento, lo que favorece una lectura limpia en un espacio reducido. Si no hay un proyecto abierto, en lugar de un panel vacío se muestra un mensaje que indica cómo proceder. El *Explorer* también permite trabajar directamente con los archivos. Con un doble clic, el usuario abre un archivo en el programa asociado del sistema, y mediante el menú contextual puede renombrar, copiar, pegar, eliminar y crear carpetas. También puede mover archivos entre carpetas arrastrándolos. Para evitar pérdidas accidentales, la eliminación solicita confirmación. Así, los datos y las exportaciones se organizan sin salir de la aplicación facilitando su usabilidad.

Results. El panel *Results* se diseñó para consultar las mediciones sin abandonar el análisis en curso, ya que en la versión anterior de GammaLab el usuario debía cambiar constantemente de pestaña para poder consultar los resultados de sus mediciones. Al ubicarse debajo del área de trabajo, permite ver las tablas y la señal al mismo tiempo, sin cambiar de pantalla. Su altura se ajusta arrastrando el borde superior, según se quiera dar prioridad a los datos o a la señal. Las mediciones de pendiente y de amplitud se separan en las pestañas *Slope* y *Amplitude*, y su contenido se actualiza cada vez que se abre el panel. La cabecera reúne el título, un menú para exportar la tabla activa a CSV o Excel y un botón de cierre, de modo que las acciones sobre los resultados quedan junto a ellos. El panel también puede cerrarse pulsando de nuevo

su icono en la barra lateral.

5.4 Tecnologías

[Escriba aquí el contenido de esta sección.]

6 DESARROLLO DE LA SOLUCIÓN

6.1 Implementación

[Escriba aquí el contenido de esta sección.]

6.2 Producto

[Escriba aquí el contenido de esta sección.]

6.2.1 Funcionalidad 1

[Escriba aquí el contenido de esta sección.]

6.2.2 Funcionalidad 2

[Escriba aquí el contenido de esta sección.]

6.2.3 Funcionalidad 3

[Escriba aquí el contenido de esta sección.]

7 PRUEBAS DE DESARROLLO

7.1 Metodología aplicada

[Escriba aquí el contenido de esta sección.]

7.2 Entorno y herramientas

[Escriba aquí el contenido de esta sección.]

7.3 Organización y documentación

[Escriba aquí el contenido de esta sección.]

8 RESULTADOS

8.1 Resumen de pruebas unitarias

[Escriba aquí el contenido de esta sección.]

8.2 Observaciones y hallazgos

[Escriba aquí el contenido de esta sección.]

8.3 Resumen de pruebas comparativas contra el sistema de referencia

[Escriba aquí el contenido de esta sección.]

8.4 Observaciones y hallazgos

[Escriba aquí el contenido de esta sección.]

8.5 Resumen de pruebas de aceptación del usuario

[Escriba aquí el contenido de esta sección.]

8.6 Observaciones y hallazgos

[Escriba aquí el contenido de esta sección.]

9 CONCLUSIONES

9.1 Análisis de Impacto del Proyecto

[Escriba aquí el contenido de esta sección.]

9.1.1 A corto, mediano y largo plazo

[Escriba aquí el contenido de esta sección.]

9.1.2 Impacto en Ingeniería de Sistemas

[Escriba aquí el contenido de esta sección.]

9.1.3 Impacto en Contextos Globales, Económicos, Ambientales y Sociales

[Escriba aquí el contenido de esta sección.]

9.2 Conclusiones y Trabajo Futuro

[Escriba aquí el contenido de esta sección.]

9.2.1 Conclusiones

[Escriba aquí el contenido de esta sección.]

9.2.2 Trabajo futuro

[Escriba aquí el contenido de esta sección.]

10 REFERENCIAS

Referencias

[1] A. K. Singh y S. Krishnan, "Trends in EEG signal feature extraction applications," *Frontiers in Artificial Intelligence*, vol. 5, art. 1072801, ene. 2023, doi: 10.3389/frai.2022.1072801. [En línea]. Disponible: https://pmc.ncbi.nlm.nih.gov/articles/PMC9905640/.

[2] P. Walla, "The Power of Time: Editorial on the Advantages of Electroencephalography (EEG) and Event-Related Potentials (ERPs) in Affective and Cognitive Neuroscience," *Brain Sciences*, vol. 15, n.º 10, art. 1054, sep. 2025, issn: 2076-3425, doi: 10.3390/brainsci15101054. [En línea]. Disponible: https://www.mdpi.com/2076-3425/15/10/1054.

[3] K. Jayalaksshme Srinivasan, P. Periasamy, y S. Gunasekaran, "Motor Imagery and Motor Execution: A Narrative Review of Electroencephalographic (EEG) Signatures, Methodological Consistency, and Translational Applications," *Cureus*, vol. 17, n.º 9, art. e93011, sep. 2025, doi: 10.7759/cureus.93011. [En línea]. Disponible: https://pmc.ncbi.nlm.nih.gov/articles/PMC12550537/.

[4] A. Chaddad, Y. Wu, R. Kateb, y A. Bouridane, "Electroencephalography Signal Processing: A Comprehensive Review and Analysis of Methods and Techniques," *Sensors*, vol. 23, n.º 14, art. 6434, ene. 2023, issn: 1424-8220, doi: 10.3390/s23146434. [En línea]. Disponible: https://www.mdpi.com/1424-8220/23/14/6434 (Visitado: 28-04-2026).

[5] M. W. J. van Es, C. Gohil, A. J. Quinn, y M. W. Woolrich, "osl-ephys: a Python toolbox for the analysis of electrophysiology data," *Frontiers in Neuroscience*, vol. 19, art. 1522675, feb. 2025, doi: 10.3389/fnins.2025.1522675. [En línea]. Disponible: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2025.1522675/full.

[6] S. Asencio Rodríguez, M. F. González Valencia, J. E. Martínez Clavijo, y G. V. Villalobos Rocha, "Gamma Lab," Memoria de trabajo de grado, Programa de Ingeniería de Sistemas, Facultad de Ingeniería, Pontificia Universidad Javeriana, Bogotá, D.C., Colombia, nov. 2025.

[7] C. Gauthier-Umaña, M. Valderrama, A. Múnera, y M. O. Nava-Mesa, "BOARD-FTD-PACC: a graphical user interface for the synaptic and cross-frequency analysis derived from neural signals," *Brain Informatics*, vol. 10, art. 12, may. 2023, issn: 2198-4026, doi: 10.1186/s40708-023-00191-x. [En línea]. Disponible: `https://link.springer.com/article/10.1186/s40708-023-00191-x`.

[8] R. T. Canolty, E. Edwards, S. S. Dalal, M. Soltani, S. S. Nagarajan, H. E. Kirsch, M. S. Berger, N. M. Barbaro, y R. T. Knight, "High gamma power is phase-locked to theta oscillations in human neocortex," *Science*, vol. 313, n.º 5793, pp. 1626–1628, sep. 2006, doi: 10.1126/science.1128115.

[9] T. E. Özkurt y A. Schnitzler, "A critical note on the definition of phase-amplitude cross-frequency coupling," *Journal of Neuroscience Methods*, vol. 201, n.º 2, pp. 438–443, 2011, doi: 10.1016/j.jneumeth.2011.08.014.

[10] J. M. Lilly y S. C. Olhede, "Generalized Morse wavelets as a superfamily of analytic wavelets," *IEEE Transactions on Signal Processing*, vol. 60, n.º 11, pp. 6036–6041, 2012, doi: 10.1109/TSP.2012.2210890.

[11] M. Vinck, M. van Wingerden, T. Womelsdorf, P. Fries, y C. M. A. Pennartz, "The pairwise phase consistency: a bias-free measure of rhythmic neuronal synchronization," *NeuroImage*, vol. 51, n.º 1, pp. 112–122, 2010, doi: 10.1016/j.neuroimage.2010.01.073.

[12] J. Aru *et al.*, "Untangling cross-frequency coupling in neuroscience," *Current Opinion in Neurobiology*, vol. 31, pp. 51–61, 2015, doi: 10.1016/j.conb.2014.08.002.

[13] P. Morris y O. Calaway, "3.1 Foundations of Neural Communication," en *General Psychology*, ago. 2025. [En línea]. Disponible: `https://acd.pressbooks.pub/generalpsychology/chapter/2192/`.

[14] P. L. Nunez, "Electroencephalography (EEG)," *Reference Module in Neuroscience and Biobehavioral Psychology*, Elsevier, 2017, isbn: 9780128093245, doi: 10.1016/B978-0-12-809324-5.03049-2. [En línea]. Disponible: `https://linkinghub.elsevier.com/retrieve/pii/B9780128093245030492`.

[15] J. D. Kropotov, "Slow, Infra-Slow Potentials, and Delta Rhythms," en *Quantitative EEG, Event-Related Potentials and Neurotherapy*, Elsevier, 2009, pp. 11-28, isbn: 9780123745125, doi: 10.1016/B978-0-12-374512-5.00001-3. [En línea]. Disponible: `https://linkinghub.elsevier.com/retrieve/pii/B9780123745125000013`.

[16] "Normal EEG variants," en *Handbook of Clinical Neurology*, vol. 160, Elsevier, 2019, pp. 143-160, isbn: 9780444640321, doi: 10.1016/B978-0-444-64032-1.00009-6. [En línea]. Disponible: https://linkinghub.elsevier.com/retrieve/pii/B9780444640321000096.

[17] E. Combrisson, T. Nest, A. Brovelli, R. A. A. Ince, J. L. P. Soto, A. Guillot, y K. Jerbi, "Tensorpac: An open-source Python toolbox for tensor-based phase-amplitude coupling measurement in electrophysiological brain signals," *PLOS Computational Biology*, vol. 16, n.º 10, art. e1008302, oct. 2020, issn: 1553-7358, doi: 10.1371/journal.pcbi.1008302. [En línea]. Disponible: https://doi.org/10.1371/journal.pcbi.1008302.

[18] M. Richards, *Software Architecture Patterns*, 2.ª ed. Sebastopol, CA, EE. UU.: O'Reilly Media, 2022. [En línea]. Disponible: https://www.oreilly.com/library/view/software-architecture-patterns/9781098134280/.

[19] EEGLAB. [En línea]. Disponible: https://sccn.ucsd.edu/eeglab/index.php.

[20] Getting started, FieldTrip. [En línea]. Disponible: https://www.fieldtriptoolbox.org/getting_started/.

[21] Introducción, Brainstorm. [En línea]. Disponible: https://neuroimage.usc.edu/brainstorm/Introduction.

[22] ISO/IEC/IEEE 29148:2018, "Systems and software engineering — Life cycle processes — Requirements engineering," 2.ª ed., International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers, nov. 2018. [En línea]. Disponible: https://www.iso.org/standard/72089.html.

[23] P. Eeles, "Capturing Architectural Requirements," Rational Software Corporation, nov. 2001. [En línea]. Disponible: https://www.ibm.com/developerworks/rational/library/4706.html.

11 ANEXOS

11.1 MRCU - Matriz de Requerimientos y Casos de Uso

[Escriba aquí el contenido de esta sección.]

11.2 PMP — *Project Management Plan*

[Escriba aquí el contenido de esta sección.]

11.3 SAD - *Software Architecture Document*

[Escriba aquí el contenido de esta sección.]

11.4 SDD - *Software Design Document*

[Escriba aquí el contenido de esta sección.]

11.5 SRS - *Software Requirements Specifications*

[Escriba aquí el contenido de esta sección.]

11.6 TD - *Test Document*

[Escriba aquí el contenido de esta sección.]

11.7 MU - Manual de Usuario

[Escriba aquí el contenido de esta sección.]

11.8 VFP - Versión Final Propuesta de Proyecto

[Escriba aquí el contenido de esta sección.]

11.9 Aplicación de escritorio

[Escriba aquí el contenido de esta sección.]